

Higher Technology Institute
10th of Ramadan City
Computer Science Department



fastAf

Fast Accurate Flawless pharmacy management system

Project Team

Moaaz Ahmed Abdulfattah	42020285
Mahmoud Attia Fathy	42021352
Mustafa Hany Mustafa	42021217
Abdulrahman Atif Abdullah	42021348
Mahmoud Mohammed Elhusseini	42021324
Anas Elsaid Abdulmaati	42021358

Supervised by
Dr: Hend Shaaban

January - May 2025

Acknowledgment

First and foremost, we thank **God** for granting us the strength, patience, and clarity to carry out this project successfully.

We would like to express our deepest gratitude to our supervisor, **Dr. Hend Shaaban**, whose continuous support, expert guidance, and constructive feedback were invaluable throughout the development of this project. Her ability to clarify complex concepts and her dedication to our success were pivotal in keeping us motivated and focused during challenging times.

We are profoundly grateful to our **professors at the Higher Technology Institute** for their mentorship. Their encouragement and commitment to academic excellence have inspired us both intellectually and personally.

Special thanks go to the **teaching assistants in our department**, whose professionalism and technical expertise have been instrumental in reinforcing our understanding and pushing us toward higher standards.

We also thank the **faculty and staff of the Computer Science Department** at the Higher Technology Institute for providing the academic environment and resources necessary for the successful completion of this graduation project.

Our sincere thanks go to our **families and friends** for their unwavering emotional and intellectual support. Their encouragement helped us stay resilient through every stage of this journey.

Finally, we extend our appreciation to **everyone who contributed, directly or indirectly**, to the success of this project. Your feedback, insights, and belief in our work brought this vision to life.

Abstract

Pharmacies often rely on a centralized cashier terminal to handle sales, print receipts, and update drug inventories. This centralization introduces a single point of failure that can disrupt pharmacy operations, especially during peak hours or hardware downtime.

This project presents a decentralized pharmacy management system that enables multiple pharmacy workers to perform the cashiering process simultaneously using Android devices, while managers and administrators can monitor and manage workflows through a web-based dashboard.

The system consists of two main components: a mobile application and a website. The Android application allows authenticated employees to scan drugs using the camera or search by name to manage shopping carts and finalize sales. It features real-time drug detection powered by a YOLOv11 object detection model, served through a WebSocket API. Touch interactions on detected drug boxes add the item to the cart and update the pharmacy's database upon checkout. The app also supports receipt returns and replacements, ordering out-of-stock drugs, organizing shelf placement based on expiry dates, and real-time database updates.

The web dashboard enables pharmacy owners to create and manage pharmacy branches, register employees and managers, track sales analytics, manage drug expiry thresholds, and schedule staff shifts. Each branch maintains its own isolated database to ensure operational security and data separation. Graphical insights, profit summaries, and receipt tracking allow business owners to make informed, data-driven decisions.

By decentralizing the cashiering process and integrating AI-powered object detection, this system enhances operational efficiency, reduces failure points, and modernizes pharmacy workflow management.

Table of Contents

Acknowledgment	i
Abstract	ii
Table of Figures	xii
Table of Tables	xiii
Table of Abbreviations	xiv
1 Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Project Objectives	2
1.4 Scope of the Project	2
1.5 Methodology Overview	3
1.6 Expected Outcomes	3
1.7 Organization of the Report	4
2 Chapter Two: Literature Review	5
2.1 Introduction	5
2.2 Pharmacy Management Systems	5
2.3 Point-of-Sale (POS) Systems in Pharmacies	6
2.4 Drug Expiry Tracking	6
2.5 Android-Based Pharmacy Apps	6
2.6 Real-Time Object Detection with YOLO	7
2.7 Cloud-Connected Mobile Systems	7
2.8 Related Academic Projects	8
3 Chapter Three: Methodology	9
3.1 Overview	9

3.2	Development Model: Incremental Model	9
3.3	Project Phases	9
3.3.1	Requirements Gathering	9
3.3.2	System Analysis	10
3.3.3	Design Phase.....	10
3.3.4	Implementation Phase	10
3.3.5	Integration Phase.....	11
3.3.6	Testing Phase	11
3.3.7	Deployment Phase	11
3.3.8	Documentation Phase.....	12
3.4	Tools and Technologies Used	12
4	Chapter Four: Requirements Specification.....	14
4.1	Introduction.....	14
4.2	User requirement.....	14
4.2.1	Pharmacy Owner Requirements	14
4.2.2	Branch Manager Requirements.....	15
4.2.3	Employee (Pharmacy Worker) Requirement	15
4.3	System Requirement	16
4.3.1	Functional Requirements	16
4.3.2	Non-Functional Requirements	17
5	Chapter Five: System Architecture and Design	19
5.1	Use case diagram	19
5.2	Sequence Diagram:	20
5.3	Activity Diagram – Web	21
5.4	Activity Diagram: Application.....	22

5.5	Architecture Design	23
5.5.1	High-Level Architecture	23
5.5.2	Component Architecture	24
5.5.3	Data Flow Architecture	24
5.5.4	Database Systems.....	25
5.5.5	Caching Strategy	26
5.5.6	System Components.....	27
5.6	Database Architecture	30
5.6.1	Schema Design.....	30
5.6.2	Entity Relationships	31
6	Chapter Six: UI/UX Design and System Screens	32
6.1	Introduction.....	32
6.2	USER INTERFACE DESIGN (UI/UX) - Dashboard	32
6.2.1	Design Principles	32
6.2.2	Design Colours.....	32
6.3	SCREENS AND COMPONENTS – DASHBOARD	33
6.3.1	Login Screen	33
6.3.2	Register Screen	34
6.3.3	Inventory Page	35
6.3.4	Employees Page	36
6.3.5	Sales Page	37
6.3.6	Expiry Warning Page	38
6.3.7	Branches Page.....	39
6.4	USER INTERFACE DESIGN (UI/UX) – Application	40
6.4.1	Design Principles	40

6.4.2	Design Colors.....	40
6.5	SCREENS AND WIDGETS – Application.....	41
6.5.1	Login Screen	41
6.5.2	Camera Scanner Screen	42
6.5.3	Medicine Details Screen	43
6.5.4	Invoice Screen.....	44
7	Chapter Seven: Backend services	46
7.1	Database Optimization.....	46
7.2	Search Functionality	46
7.3	API Design.....	47
7.3.1	RESTful Principles	47
7.3.2	API Endpoints Structure	47
7.3.3	Request/Response Format.....	47
7.3.4	Error Handling	48
7.4	Caching Implementation.....	48
7.4.1	Redis Architecture.....	48
7.4.2	Caching Strategy	48
7.4.3	Cache Invalidation	49
7.5	Security Architecture	49
7.5.1	Authentication Flow.....	49
7.5.2	Authorization Model.....	50
7.5.3	Data Protection.....	50
7.5.4	API Security.....	51
7.6	Deployment Architecture	51
7.6.1	CI/CD Pipeline.....	51

7.6.2	Infrastructure Setup.....	51
7.6.3	Containerization.....	52
7.6.4	Cloud Resources	53
7.7	Scalability Considerations	53
7.7.1	Horizontal Scaling	53
7.7.2	Database Scaling.....	54
7.7.3	Cache Scaling.....	54
7.8	Monitoring & Observability	54
7.8.1	Logging Strategy.....	54
7.8.2	Performance Monitoring.....	55
7.8.3	Alert System.....	55
7.9	Testing Strategy	55
7.9.1	Unit Testing.....	55
7.9.2	Integration Testing	56
7.9.3	Performance Testing	56
7.10	Appendices.....	57
7.10.1	Database Schema Diagram	57
7.10.2	Architecture Diagrams	58
7.10.3	API Documentation Reference	58
8	Chapter Eight: FrontEnd and Application implementation	59
8.1	introduction.....	59
8.2	Main Page Component.....	59
8.3	StackedFeatureCard	63
8.4	TestimonialSection.....	65
8.5	6. FrequentlyAsked.....	67

7.Footer	68
8.5.1 Tools & Libraries Used:	70
8 . 6 1.The signin.....	70
8.6.1 Key Features:	71
8.6.2 Component Breakdown:	71
8.7 2- The sign up	73
8.7.1 Breakdown of the sign up Component:	74
8.7.2 Key Features:	76
8.7.3 Possible Improvements:	76
8.8 1-CreateEmployeeForm.....	77
8.8.1 EditEmployeeForm	77
8.8.2 DeleteEmployee	78
8.8.3 ShiftsCard	78
8.8.4 DataTableRowActions	78
8.8.5 Shifts and Employees Data Fetching	79
8.8.6 Pharmacy Shifts and Employee Management	79
8.8.7 ExpiryPage Component	79
8.8.8 Columns Configuration for the Data Table.....	80
8.8.9 header	81
8.8.10 CommandMenu Component.....	81
8.9 inventory	82
8.9.1 Columns Definition.....	82
8.9.2 Filtering Logic:	82
8.9.3 Inventory Page with Dynamic Data Fetching:	83
8.10 5-pharmacy	83

8.10.1	DeletePharmacy	83
8.10.2	DeleteShift	83
8.10.3	EditPharmacyForm	83
8.10.4	PharmacyItem	84
8.10.5	PharmacyItemWithMenu	84
8.10.6	ShiftItem	84
8.10.7	ShiftItemWithContext	84
8.11	Android implementation	84
8.11.1	What is Android?	84
8.11.2	Why Android?	85
8.11.3	APPLICATION ARCHITECTURE :	85
8.11.4	Advantages of This Architecture :	88
8.11.5	STATE MANAGEMENT :	89
8.11.6	How It Works?	91
8.11.7	USER INTERFACE DESIGN USING XML:	92
8.11.8	Architecture :	102
8.11.9	UI Layer :	102
8.11.10	Data Flow :	104
8.11.11	Functionality :	104
9	Chapter Nine: Drug Detection Model Development and Deployment.....	105
9.1	Introduction.....	105
9.2	Dataset Collection and Preprocessing.....	105
9.2.1	Scraping the Egyptian Drug Database	105
9.2.2	Initial Image Gathering via SERPAPI	105
9.2.3	Manual Photographic Data Collection.....	106

9.2.4	Annotation and Augmentation with Roboflow	106
9.3	Model Training and Selection.....	106
9.3.1	Training Configurations.....	107
9.3.2	Export to ONNX.....	107
9.4	WebSocket API Server Implementation	107
9.4.1	Technology Stack.....	107
9.4.2	Server Workflow	108
9.4.3	Robust Error Handling.....	108
9.4.4	Logging and Monitoring	109
9.5	Containerization and Deployment	110
9.5.1	Dockerization.....	110
9.5.2	Cloud Deployment.....	110
9.5.3	Integration with Mobile Application.....	110
9.6	Yolo model results and accuracy	111
9.7	Some validation outputs.....	112
10	Chapter Ten: Limitations and Challenges	113
10.1	Introduction.....	113
10.2	Technical Limitations.....	113
10.2.1	YOLO Model Detection Accuracy	113
10.2.2	Model Deployment Constraints	113
10.2.3	Device Compatibility	113
10.2.4	Database Synchronization.....	114
10.3	Design and Architecture Challenges.....	114
10.3.1	Multi-Branch Data Isolation	114
10.3.2	Real-Time Collaboration.....	114

10.3.3	Barcode/Visual Redundancy	114
10.4	User Experience Challenges	114
10.4.1	Training Staff to Use the App	114
10.4.2	Receipt Return/Replacement Logic	114
10.5	Operational and Deployment Limitations.....	115
10.5.1	Internet Dependency	115
10.5.2	Scalability Concerns	115
10.5.3	Security Risks	115
11	Chapter Eleven: Conclusion and Future Work	116
11.1	Conclusion	116
11.2	Future Work	116
	References.....	118

Table of Figures

Figure 5-1 use case diagram	19
Figure 5-2 Sequence Diagram	20
Figure 5-3 5.3Activity Diagram – Web	21
Figure 5-4 5.4Activity Diagram: Application	22
Figure 5-5 heigh level architecture	23
Figure 5-6 cashing strategy diagram	27
Figure 6-1 login screen	33
Figure 6-2 signup screen	34
Figure 6-3 inventory screen	35
Figure 6-4 Employee screen	36
Figure 6-5 Sales screen	37
Figure 6-6 receipt screen.....	37
Figure 6-7 Expiry warning screen.....	38
Figure 6-8 branches page	39
Figure 6-9 app login screen.....	41
Figure 6-10 pages navigator.....	42
Figure 6-11 search for drug info	43
Figure 6-12 cart screen.....	44
Figure 6-13 confirm order.....	45
Figure 6-14 make order.....	45
Figure 6-15expiry warning page.....	45
Figure 8-1 view model	87
Figure 9-1 yolo model output	111
Figure 9-2 model predicted levcet tablet	112
Figure 9-3 model predicted 5 different types of drugs	112

Table of Tables

Table 3-1 Tools and Technologies used in the project	12
Table 8-1front components	59
Table 8-2 tools used	60
Table 8-3front libraries used	61
Table 8-4front technologies used	61
Table 8-5 front libraries	62
Table 8-6front tools	62
Table 8-7 Behavior Description	64
Table 8-8 Behavior Description	66
Table 8-9 Technologies used.....	67
Table 8-10 behavior descriptions	68
Table 8-11 technologies used	69
Table 8-12 behavior description.....	69
Table 8-13 project structure	87

Table of Abbreviations

Abbreviation	Full Form
POS	Point-of-Sale
YOLO	You Only Look Once
API	Application Programming Interface
UI/UX	User Interface/User Experience
MVVM	Model-View-ViewModel
MVI	Model-View-Intent
REST	Representational State Transfer
JWT	JSON Web Token
CI/CD	Continuous Integration / Continuous Deployment
ONNX	Open Neural Network Exchange
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
2FA	Two-Factor Authentication
RBAC	Role-Based Access Control

1 Chapter 1: Introduction

1.1 Background

Pharmacies play a critical role in public health by distributing medications and providing essential healthcare products. The cashiering process is central to daily pharmacy operations, it's where drugs are billed, stock is updated, and sales are recorded. However, traditional cashiering systems in pharmacies depends on a single terminal or cashier point, which acts as a bottleneck. This creates a single point of failure; if the cashier is unavailable or the terminal malfunctions, the entire pharmacy workflow halts, leading to operational inefficiency and customer dissatisfaction.

To overcome this challenge, this project proposes a decentralized cashiering system that enables parallel cashier operations using Android devices. By allowing pharmacy workers—especially doctors on shift—to process sales from their smartphones, the system eliminates dependency on a single cashier, enhances operational efficiency, and minimizes service interruptions.

1.2 Problem Statement

Most pharmacies still rely on centralized cashiering systems, which suffer from several critical limitations:

- Queue buildup during peak hours.
- Complete workflow interruption if the cashier device fails or the cashier is unavailable.
- Slow stock updates and higher error rates in drug tracking.
- Inflexible access to receipts and order history.

These inefficiencies lead to revenue loss, poor customer experience, and operational inefficiencies, highlighting the need for a more robust and flexible solution.

1.3 Project Objectives

This project aims to develop a decentralized pharmacy management system with the following objectives:

- Eliminate the single point of failure by enabling cashiering from multiple Android devices.
- Provide a web-based dashboard for pharmacy owners to manage stock, branches, employees, and sales analytics.
- Integrate AI-powered drug recognition using YOLOv11 for real-time camera-based product detection.
- Automate stock tracking, including expiry date warnings and low-quantity alerts.
- Streamline receipt management, returns, and replacements.
- Optimize shelf organization based on expiry dates to minimize waste.

1.4 Scope of the Project

The project includes two main components:

- **Android Mobile Application:**
 - Role-based authentication for pharmacy employees.
 - Allows pharmacy employees to log in and perform sales transactions.
 - Uses a camera integrated with a trained YOLOv11 model to detect drugs in real-time eliminating the need of the barcode.
 - Maintains a shopping cart, checkout process, and receipt generation.
 - Supports drug search, receipt replacement and return, and real-time database updates.
 - Order placement for low-stock medications.
 - Organizes shelf placement based on drug expiry dates.
- **Web-Based Dashboard:**
 - Allows pharmacy owners to register, manage branches, and create staff accounts.
 - Sales analytics & profit insights via interactive dashboards.

- Tracks drug stock levels, expiry dates, and sales metrics.
- Manages shift schedules, branch databases, and employee permissions.
- Configurable expiry thresholds and automated warnings.

1.5 Methodology Overview

The system is developed using a modular and scalable architecture. The development methodology includes:

- Requirement gathering and system design.
- Collect the needed data using web scraping and photographic picture as needed.
- Training an object detection model for drug detection.
- Web development using modern frontend frameworks.
- Android application development with real time communication with the database and the object detection model.
- Backend development using reliable frameworks to store and manage the database.
- API development Ensuring real-time sync between mobile, web, and database.
- Testing and deployment validating functionality, and performance.

Further Details in Chapter 3

1.6 Expected Outcomes

By the end of this project, we expect to deliver:

- A fully functional Android app for decentralized cashiering and inventory real time management.
- A responsive web dashboard for business insights and administrative control.
- Seamless integration between the mobile app, cloud backend, and analytics dashboards.
- A stable object detection model capable of identifying labeled drug packages with high accuracy.

- Automated stock & expiry tracking to minimize losses.
- Enhanced in pharmacy shelve management and ordering.

1.7 Organization of the Report

- Chapter 1 introduces the project and clearly states the problem it aims to solve.
- Chapter 2 provides a literature review of existing pharmacy systems and cashiering solutions.
- Chapter 3 System Development Methodology
- Chapter 4 outlines the system requirements, user roles.
- Chapter 5 describes the overall system architecture, including database schema, design patterns, and component interactions.
- Chapter 6 focuses on the UI/UX design, including user interface principles and screenshots of the system.
- Chapter 7 details the backend logic implementations and cloud-based deployment strategies.
- Chapter 8 explains the implementation process for both the mobile application and the web system.
- Chapter 9 covers data collection, training, and deployment of the YOLO-based drug detection model.
- Chapter 10 discusses the limitations, technical challenges, and lessons learned during development.
- Chapter 11 concludes the report with a summary of findings and recommendations for future enhancements.

2 Chapter Two: Literature Review

2.1 Introduction

The efficiency and reliability of pharmacy operations rely heavily on well-designed information systems. This chapter explores prior research and real-world systems related to cashier decentralization, drug stock management, real-time object detection, and cloud-based data synchronization. The review highlights the limitations of current solutions and builds a foundation for the proposed system.

2.2 Pharmacy Management Systems

Traditional pharmacy systems focus on stock control, billing, and customer records. Solutions like PharmaOne, Liberty Software, and Marg ERP provide centralized interfaces for sales and inventory.

- These systems offer:
 - Drug catalogs with expiry tracking
 - Billing terminals with receipt printing
 - Reporting dashboards
- Limitations:
 - Most require a single cashier point per branch.
 - Lack of the ability to use other devices other than a counter computer.
 - Inflexible access to system functionality outside the counter.

These drawbacks reinforce the need for parallel cashiering on mobile devices, especially during busy hours or emergencies.

2.3 Point-of-Sale (POS) Systems in Pharmacies

POS systems in pharmacies often combine hardware (barcode scanners, terminals) with billing software. Studies such as “Improving Drug Dispensation Using Smart POS Terminals” (IEEE, 2019) report improved transaction speeds but also highlight centralization as a bottleneck.

Research from Health IT Analytics (2021) indicates that 38% of pharmacies experience downtime due to POS or cashier device issues, leading to delays in prescriptions and order fulfillment.

Our project FastAF addresses this by:

- Decentralizing cashier roles.
- Using commodity smartphones instead of dedicated POS hardware.

2.4 Drug Expiry Tracking

Several systems include expiry alerts to ensure safety and compliance. These are usually based on threshold values (e.g., 3 months before expiry). Works like “Design of an Expiry Management System for Pharmacies” (IJCA, 2020) emphasize proactive warning systems and first-expiry-first-out (FEFO) inventory practices.

Our system extends this by:

- Allowing managers to set custom expiry thresholds.
- Recommending shelf organization based on expiry proximity.

2.5 Android-Based Pharmacy Apps

Mobile apps in the pharmacy space are mostly customer-facing (e.g., PillPack, CVS, MyPharmacyApp). Backend pharmacy tools on Android are rare and often lack:

- Real-time object detection
- Integration with local stock data

Studies such as “Mobile Applications for Pharmacy Inventory” (Elsevier, 2022) emphasize the need for pharmacist-friendly UIs and rapid data syncing with centralized systems.

Our solution addresses this with:

- Cart system
- Receipt management

2.6 Real-Time Object Detection with YOLO

YOLO (You Only Look Once) models are state-of-the-art for object detection. YOLOv11, in particular, is efficient for mobile and real-time inference. Research confirms its accuracy for box-based drug identification in retail contexts.

Key advantages of YOLOv11:

- Fast and lightweight (suitable for mobile-cloud setups)
- High accuracy with small datasets
- Easy to retrain with custom drug images

Our use of YOLOv11 allows:

- Real-time camera detection of drugs
- Touch-based drug selection and info retrieval
- Red-green visual feedback based on user interaction

2.7 Cloud-Connected Mobile Systems

Cloud-connected systems using REST or WebSocket APIs allow real-time communication between devices and central databases. Firebase, AWS, and Supabase are commonly used platforms. Studies such as “WebSocket vs REST for Real-Time Communication” (IEEE, 2020) support WebSockets for low-latency scenarios like live detection.

Our implementation uses:

- WebSocket API for real-time drug prediction
- Cloud database per branch to isolate data

- Role-based access with secure token login

2.8 Related Academic Projects

Several academic projects attempt to modernize pharmacy systems, such as:

- Smart Pharmacy Inventory Management (2021): Focused on RFID for stock detection.
- Mobile POS for Healthcare Products (2020): Proposed offline POS terminals, but lacked AI and camera support.
- DrugShelf: A Prescription-to-Purchase App (2022): Combined OCR with medicine ordering.

Our project advances by:

- Blending real-time AI vision with business logic
- Supporting multiple branches and role hierarchies
- Creating a seamless Android-to-Web system

3 Chapter Three: Methodology

3.1 Overview

This chapter outlines the methodology followed in developing the decentralized pharmacy management system FastAF. A structured engineering approach was adopted to ensure reliability, scalability, and usability. The development lifecycle chosen for this project is the Incremental Software Development Model, enabling parallel and iterative development of major modules: the Android application, the website, and the AI object detection system.

3.2 Development Model: Incremental Model

The incremental model was selected for the following reasons:

- Allows the core system to be developed in parts.
- Enables continuous feedback and testing during each increment.
- Supports early deployment of usable subsystems (e.g., cashiering via mobile before full web dashboard is ready).
- Reduces risk by tackling complex parts (e.g., real-time AI drug detection) separately.

3.3 Project Phases

3.3.1 Requirements Gathering

- Identified stakeholders: pharmacy owners, branch managers, workers.
- Understand pain points with centralized cashiering used on the pharmacies and validate the information by questionnaire a pharmacy staff with the findings.
- Studied existing systems to identify functional and non-functional gaps.
- Collected technical requirements for mobile-first and AI-driven features.
- Understand the data needed to be viewed for the pharmacy owners and decision-makers.

3.3.2 System Analysis

- Created use case diagrams to represent interactions between users and the system.
- Defined user roles and permissions.
- Analyzed technical constraints such as Android compatibility, database isolation, and real-time camera integration.

3.3.3 Design Phase

- Developed UML models including:
 - Use case diagrams
 - Activity diagrams
 - Class diagrams
- Designed database schema to handle:
 - Multi-branch inventory
 - Drug details
 - Expiry tracking
 - Inventory tracking
 - Sales and receipts
- Created wireframes and UI/UX flows for mobile and web platforms.
- Chose design patterns.

3.3.4 Implementation Phase

- **Mobile App:** Developed using Kotlin with XML-based UI, incorporating CameraX API (via WebSocket), Retrofit/OkHttp for networking, Coroutines for asynchronous operations, and MVVM architecture with LiveData, DataBinding, and ViewBinding for reactive UI updates.
- **Web System:** Built using React (frontend).
- **Cloud Based Backend:** developed and implemented with love

- **Object Detection Model:** Yolo 11 model, trained on custom drug images, integrated through a FastAPI WebSocket for real-time object detection.
- Integrated login and role-based access control.

3.3.5 Integration Phase

- The Android application communicates with the backend through RESTful APIs to perform sales, orders, and inventory updates.
- The Android application connected to the object detection system via WebSocket connection for real-time predictions.
- Upon checkout, the app sends sales data to the backend, which updates the cloud-hosted database in real time.
- The backend ensures data consistency across all branches and synchronizes the updated inventory for all authenticated users.
- Order requests submitted from the app are stored in the database and viewed to all authenticated users.
- Once a delivery is arrived and confirmed via the app the inventory will be updated.
- The website dashboard fetches real-time data on inventory, sales, and expiry warnings to provide up-to-date insights.
- All transactions are logged with timestamps, employee IDs, and branch IDs to ensure traceability and accurate reporting.

3.3.6 Testing Phase

- Conducted unit testing on all modules (e.g., login, drug detection, receipt generation).
- Performed integration testing between app, backend, and AI service.

3.3.7 Deployment Phase

- Deployed backend on Railway Cloud with persistent storage.
- Deployed YOLO model on a separate instance to reduce latency and isolate failures.
- Delivered Android APK to pharmacy staff and hosted the web system on a domain.

3.3.8 Documentation Phase

Created this comprehensive project report including system requirements, architecture, design models, source code overview, and potential future work for improvements.

3.4 Tools and Technologies Used

Table 3-1 Tools and Technologies used in the project

Tool/Technology	Purpose
Yolov11m	Object detection model
FastAPI	WebSocket API
Roboflow	Data annotation and dataset versioning
serpapi	Google image web scraping
selenium	Web scraping library
Figma	UI/UX design
Photoshop	Photo creating and editing
Git & GitHub	Version control
Android Studio Code	Primary IDE for Android app development.
Layout Editor	Visual editor for designing XML layout files (drag-and-drop).
Gradle	Build project, dependency management, and packaging APKs.
CameraX	Captures real-time camera frames for drug detection.
Retrofit	Handles HTTP API communication.
Room	Provides local database support and caching on the device.
React	JavaScript library for building user interfaces.
Vite	Development server and build tool for React + TypeScript projects.
TailwindCSS	Utility-first CSS framework for rapidly styling UI components.
TypeScript	Superset of JavaScript that provides static typing.
Shadcn	Pre-built, styled UI components built with TailwindCSS and Radix.
React Router	Enables client-side routing in the React app.
TanStack Query	Manages server state, caching, and data fetching in React.
Framer Motion	Used for creating smooth animations and UI transitions.

Zod	Schema-based validator and parser for TypeScript data validation.
Zustand	Scalable state management library for React applications.

4 Chapter Four: Requirements Specification

4.1 Introduction

This chapter outlines the essential requirements necessary for developing and deploying the decentralized pharmacy management system. The requirements specification is divided into user and system categories.

System requirements are divided into functional and non-functional sub-categories. Understanding these requirements ensures that the system operates efficiently, securely, and meets user expectations.

4.2 User requirement

4.2.1 Pharmacy Owner Requirements

- Pharmacy owner should be able to create a pharmacy profile and register multiple branches.
- Pharmacy owner should be able to create manager and employee accounts for each branch.
- Pharmacy owner should be able to give access to some employee accounts to be able to view and order stock shortage.
- Pharmacy owner should be able to log into a web dashboard and view sales analytics per branch.
- Pharmacy owner should be able to monitor each drug's stock, expiry, revenue, and profit.
- Pharmacy owner should be able to set expiry thresholds to get automatic warnings for soon-to-expire drugs.
- Pharmacy owner should be able to assign and manage employee shifts.
- Each branch to have its own separate database for better control and isolation.
- Pharmacy owner should be able to view past receipts for audits and insights.

4.2.2 Branch Manager Requirements

- Branch Managers should be able to log into the web system with access limited to their branch.
- Branch Managers should be able to register employees to work in their specific branch.
- Branch Managers should be able to monitor the activity and shifts of employees under their management.
- Branch Managers should be able to track sales, receipts, inventory, near expiry and profits in real-time for decision-making.

4.2.3 Employee (Pharmacy Worker) Requirement

- Employee should be able to log into the Android app using my branch-assigned account.
- Employee should be able to search for a drug by name or scan it using my phone camera.
- Employee should be able to add drugs to a cart, adjust quantity in packs and units, and complete a sale.
- The system should be able to automatically print a receipt after each transaction.
- Employee should be able to return or replace a receipt by selecting it from receipt history as long as it is during 14 days from the purchase day.
- Authorized employee should be able to place orders for out-of-stock or low-stock drugs.
- Employee should be able to input delivery details for incoming orders (expiry date and cost).
- Employee should be able to see how to arrange new stock on the shelf based on expiry date.
- Employee should be able to receive alerts about drugs nearing expiration.
- Employees want the camera system to help them recognize drugs without the need of the barcode.

4.3 System Requirement

4.3.1 Functional Requirements

Functional requirements describe the core features and behaviors expected from the system.

Website Functional Requirements

- Allow pharmacy owners to sign up, sign in, create pharmacy profiles, and add branches.
- Enable login access for managers to specific branches.
- Provide a dashboard with visualized sales data, revenues, and profits.
- Enable drug stock management, including tracking expiry dates, costs, and quantities.
- Allow receipt tracking and history viewing.
- Issue expiry warnings based on a configurable threshold (e.g., 3 months).
- Enable employee account creation, assigning them as workers or branch managers.
- Allow shift scheduling and staff information management.

Android Application Functional Requirements

- Allow pharmacy workers to log in to their respective branch.
- Support manual drug information search or camera-based detection using an AI model.
- Add detected drugs to the cart using touch interactions.
- Allow quantity input by packs and units and perform checkout operations.
- Print receipts and update inventory in real time.
- Allow under 14 days receipt return and replacement operations.
- Show stock shortage and enable ordering.
- Log delivery details (cost and expiry) when new stock arrives.
- Display drug shelf arrangement based on expiry dates.
- Display an expiry warning page.

4.3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes, performance benchmarks, and constraints of the system. These ensure the system is usable, reliable, and scalable.

- **Performance Requirements**
 - The system should handle multiple concurrent users across different branches without performance degradation.
 - Drug detection results from the YOLO model must return within 500 milliseconds on average over a stable internet connection.
 - Web dashboard pages should load in under 3 seconds on standard hardware.
- **Usability Requirements**
 - The Android application and website must have intuitive interfaces for users with limited technical experience.
 - The user interface must support touch interactions for mobile users and standard mouse/keyboard input for web users.
- **Availability Requirements**
 - The system must be available 24/7 with a minimum 99.5% uptime guarantee.
- **Scalability Requirements**
 - The architecture must support the addition of new pharmacy branches without service interruption.
 - The system must be able to scale horizontally to support growing data and users over time.
- **Security Requirements**
 - Users must authenticate via secure login using passwords and unique branch-based credentials.
 - Data transmission between app, website, and backend must be encrypted.
 - Each pharmacy branch must have isolated access to its own database and not see other branches' data.
 - User roles must be enforced (e.g., owners, managers, workers) with role-based access control.

- **Maintainability Requirements**

- The system must follow modular design principles to simplify debugging and future updates.
- Code must be well-documented and follow consistent naming conventions and standards.
- Logs must be generated for all major events (e.g., sales, inventory changes, login attempts) for audit purposes.

- **Reliability Requirements**

- The database must ensure transactional integrity during cart checkouts to prevent stock mismatch.
- System processes must recover from unexpected failures and restart services automatically when needed.

- **Portability Requirements**

- The mobile application must be compatible with devices running Android 8.0 (Oreo) or higher.
- The web interface should be accessible from any modern browser (Chrome, Firefox, Edge, Safari).

5 Chapter Five: System Architecture and Design

5.1 Use case diagram

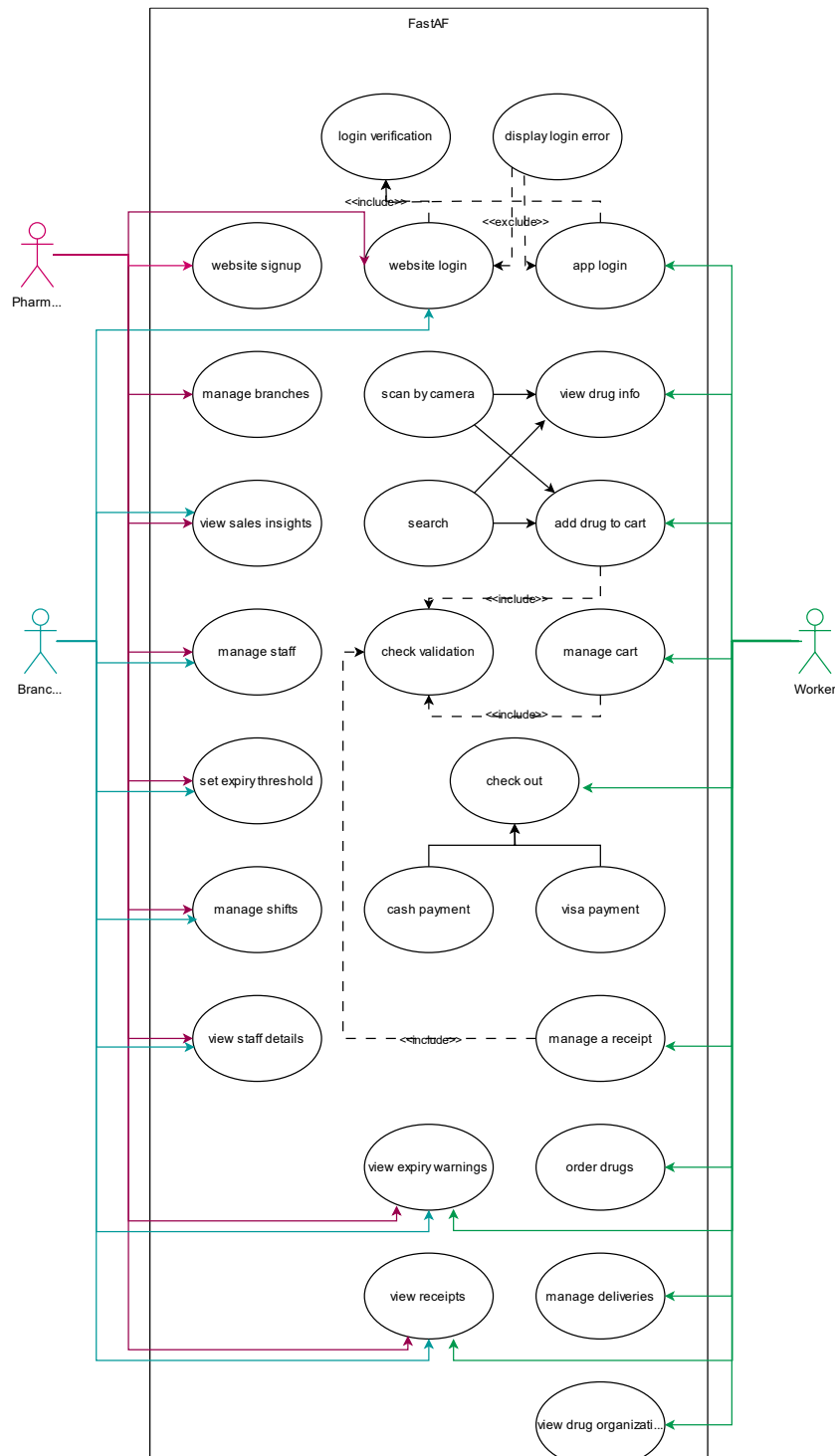


Figure 5-1 use case diagram

5.2 Sequence Diagram:

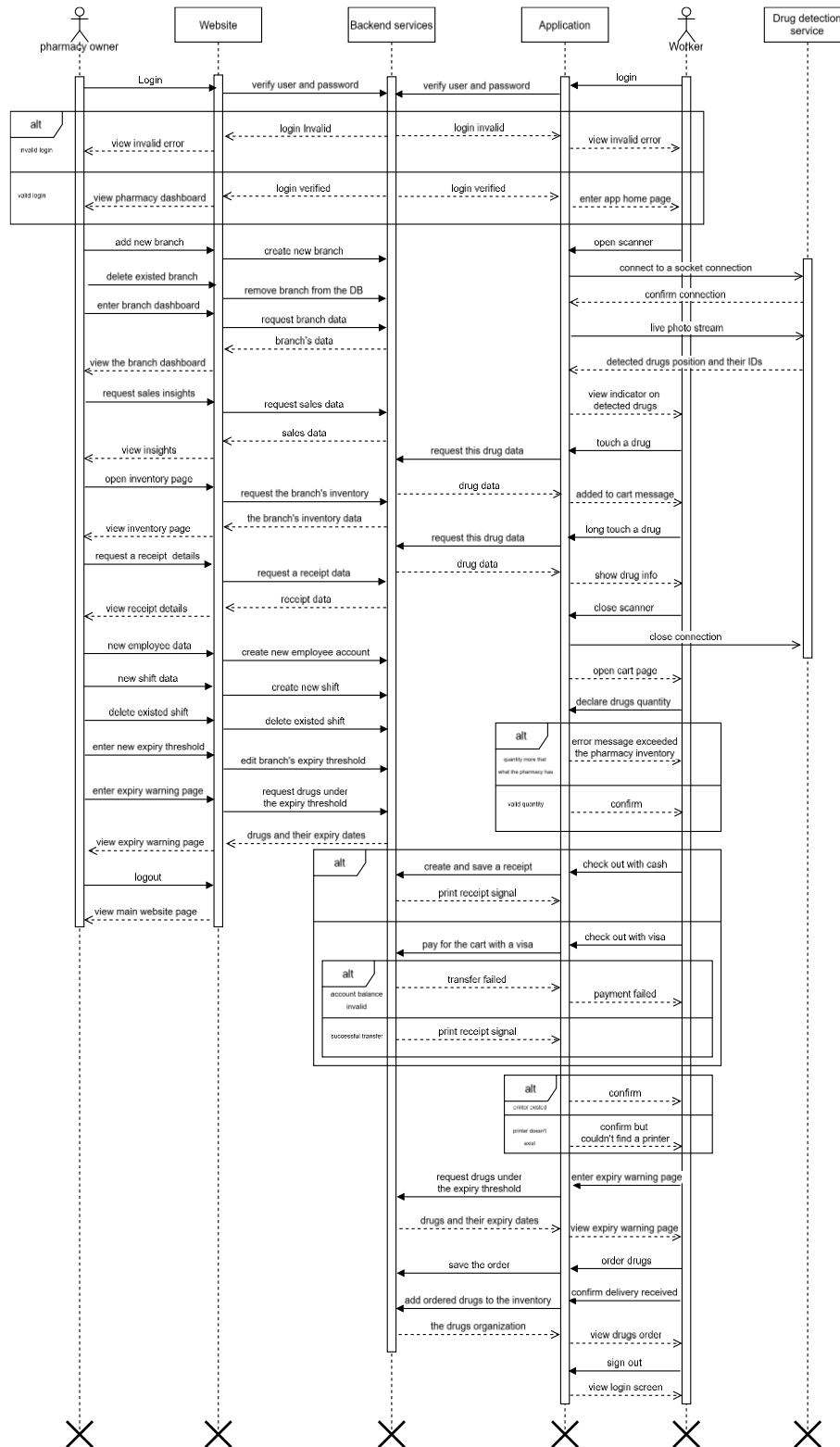


Figure 5-2 Sequence Diagram

5.3 Activity Diagram – Web

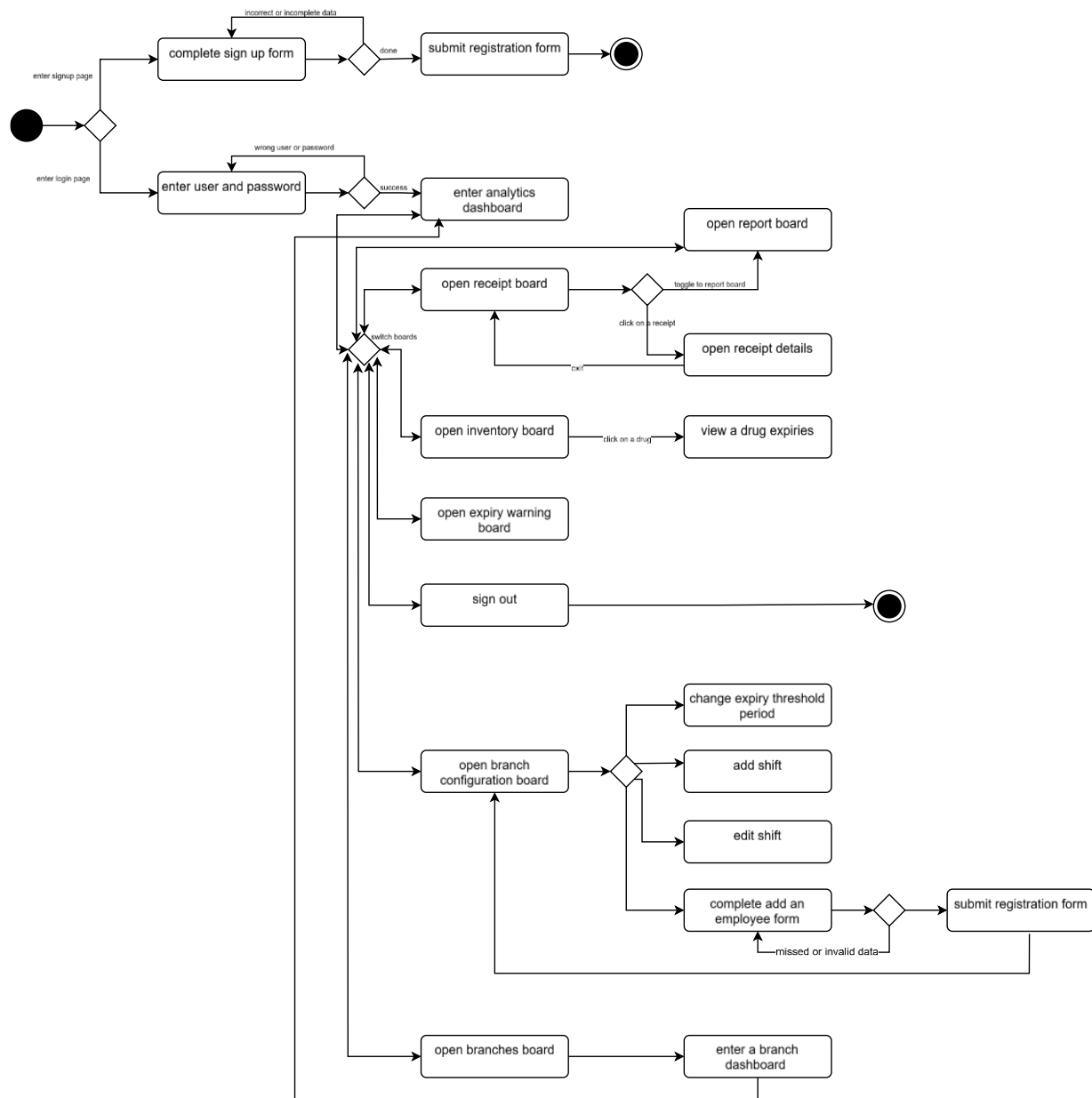


Figure 5-3 5.3Activity Diagram – Web

5.4 Activity Diagram: Application

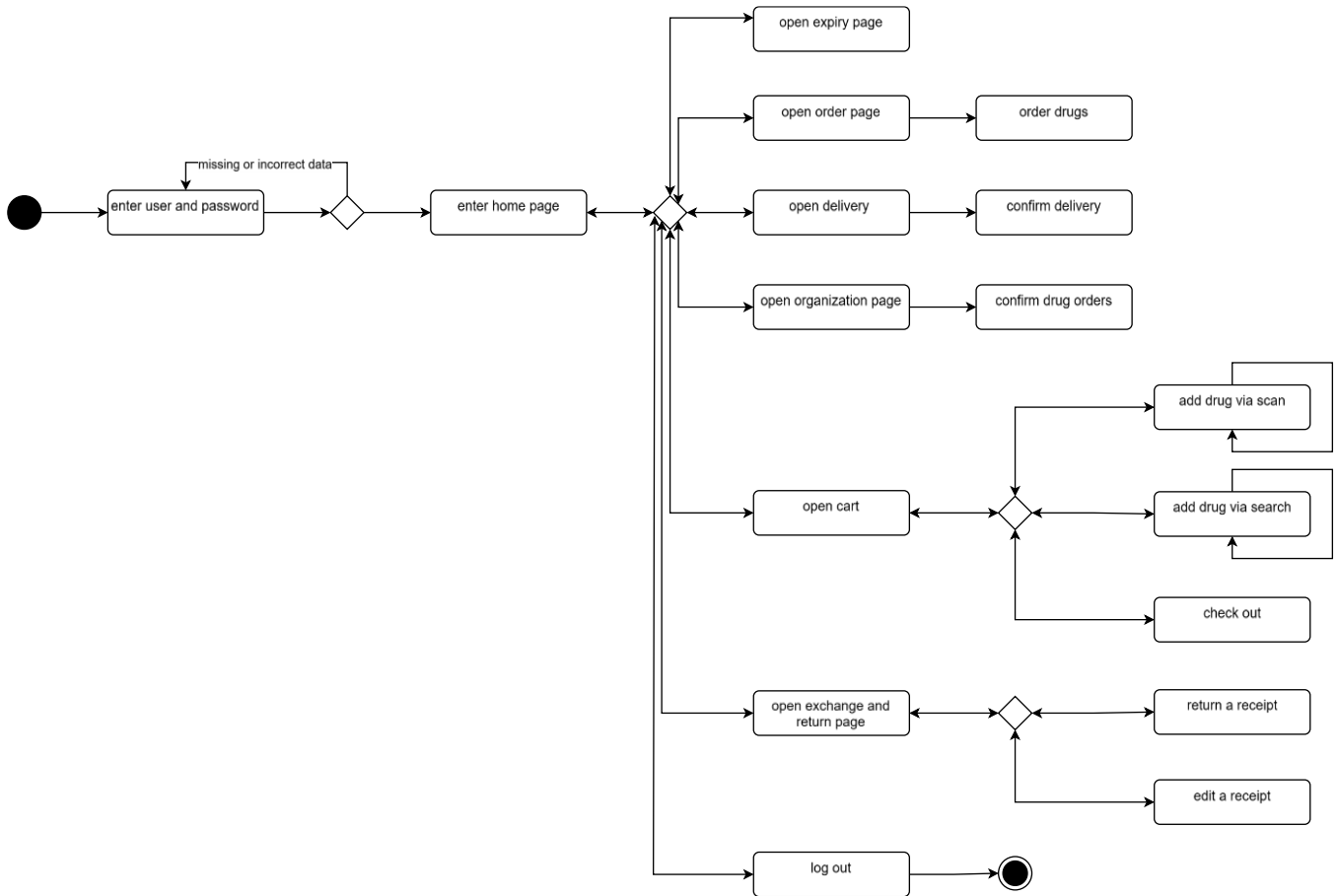


Figure 5-4 5.4Activity Diagram: Application

5.5 Architecture Design

5.5.1 High-Level Architecture

FastAF employs a traditional N-tier architecture pattern consisting of the following layers:

1. **Presentation Layer:** While not directly included in the backend codebase, this layer represents client applications (web, mobile) that consume the API
2. **API Layer:** RESTful API endpoints that handle HTTP requests and responses
3. **Service Layer:** Core business logic implementation
4. **Data Access Layer:** Database interaction logic
5. **Database Layer:** PostgreSQL for persistent storage and Redis for caching

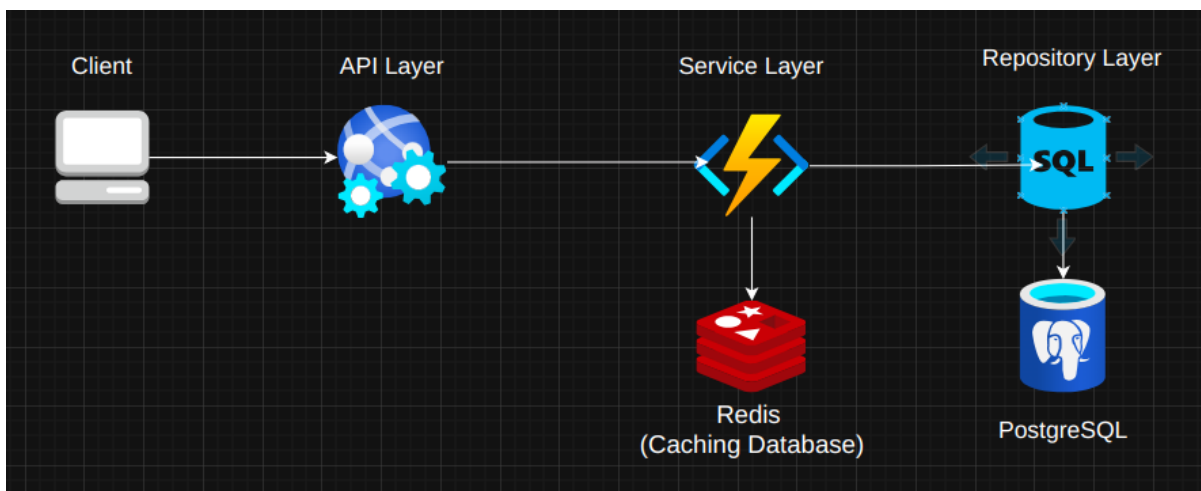


Figure 5-5 High level architecture

This layered architecture provides several benefits:

- Clear separation of concerns
- Improved maintainability
- Flexibility to replace components
- Ability to scale components independently

The communication between layers follows a strict top-down approach, where higher layers can only interact with the immediate layer below, preventing tight coupling and promoting modularity.

5.5.2 Component Architecture

The system is organized into several functional components:

- **Authentication Component:** Handles user authentication, session management, and token validation
- **Pharmacy Management Component:** Manages pharmacy data and operations
- **Inventory Component:** Tracks drug inventory across pharmacies
- **Employee Management Component:** Manages pharmacy staff and their roles
- **Sales Component:** Processes sales transactions and receipts
- **Order Management Component:** Handles drug ordering process
- **Reporting Component:** Generates operational reports

Each component consists of controllers, services, repositories, and data transfer objects (DTOs), forming a vertical slice of functionality that can be developed and maintained independently.

5.5.3 Data Flow Architecture

Data flows through the system in a well-defined manner:

1. **Request Handling:** API controllers receive HTTP requests from clients
2. **Authentication & Authorization:** Security filters validate user credentials and permissions

3. **Business Logic Processing:** Service layer processes the request according to business rules
4. **Data Access:** Repository layer fetches or modifies data in the database
5. **Caching:** Frequently accessed data is cached in Redis to improve performance
6. **Response Generation:** Controllers transform the processed data into appropriate HTTP responses

This flow ensures that each request is handled efficiently while maintaining proper separation between concerns.

- 4. Technology Stack
- 4.1 Backend Framework

FastAF is built on Spring Boot, which provides a comprehensive framework for building Java-based enterprise applications. Key Spring components used include:

- **Spring MVC:** For building the RESTful API endpoints
- **Spring Data JPA:** For database access and ORM capabilities
- **Spring Security:** For authentication and authorization
- **Spring Boot Actuator:** For monitoring and managing the application

Spring Boot was chosen for its robust ecosystem, excellent documentation, and strong community support. It significantly reduces boilerplate code and provides built-in solutions for common challenges in enterprise application development.

5.5.4 Database Systems

PostgreSQL

PostgreSQL serves as the primary database for persistent storage. It was chosen for:

- Strong ACID compliance
- Advanced data types (including geospatial support for pharmacy locations)

- Full-text search capabilities
- Excellent performance and reliability
- Open-source with strong community support

The database schema is designed with normalization principles in mind, optimized with appropriate indexes, and enhanced with PostgreSQL-specific features like materialized views and stored procedures where beneficial.

- Redis

Redis is utilized as a caching layer to improve system performance. It stores frequently accessed data such as:

- User session information
- Frequently queried inventory data
- Configuration settings
- Recently accessed pharmacy data

Redis was selected for its exceptional performance, in-memory data structure store capabilities, and built-in support for data structures like lists, sets, and hashes that simplify caching implementation.

5.5.5 Caching Strategy

The caching strategy is designed to balance between performance improvements and data consistency:

- **Cache-Aside Pattern:** The system checks the cache first for data; if not found, it retrieves from the database and populates the cache

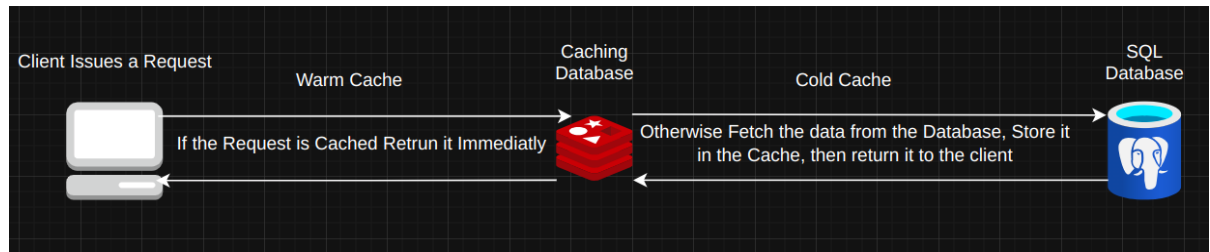


Figure 5-6 caching strategy diagram

- **Time-Based Expiration:** Cached data expires after a predefined period to prevent stale data issues
- **Selective Caching:** Only frequently accessed, relatively static data is cached to maximize cache efficiency
- **Cache Invalidation:** Clear cache entries when the underlying data changes

This strategy significantly reduces database load and improves response times for frequently accessed data.

5.5.6 System Components

Controller Layer

The controller layer serves as the entry point for API requests and is responsible for:

- Request validation
- Routing to appropriate service methods
- Response formatting
- Basic error handling

Controllers are organized by domain entities (Pharmacy, Drug, Employee, etc.) and follow RESTful principles. They use DTOs (Data Transfer Objects) for input and output, providing a clear contract for API consumers and preventing direct exposure of domain entities.

Each controller method is annotated with Swagger documentation to generate comprehensive API documentation. Input validation is performed using Jakarta Validation annotations, ensuring that requests meet the expected format before being processed further.

Example controller responsibilities include:

- PharmacyController: Manages pharmacy CRUD operations, staff management, and inventory
- ReceiptController: Handles sales transactions and receipt generation
- OrderController: Manages drug ordering processes
- UserController: Handles user account operations

Service Layer

The service layer contains the core business logic and orchestrates operations that may span multiple repositories. Key responsibilities include:

- Implementing business rules and validations
- Transaction management
- Coordinating multiple database operations
- Managing cache operations
- Handling business exceptions

Services are designed to be stateless, making them thread-safe and suitable for concurrent access. They're organized by functional domains rather than specific entities, allowing for better encapsulation of related operations.

Complex workflows, such as processing a sale or creating an order, are broken down into smaller, more manageable methods within the service classes. This approach improves code readability and makes testing more straightforward.

Key services include:

- PharmacyService: Core pharmacy operations including inventory management
- EmployeeService: Employee management and authorization
- ReceiptService: Sales and receipt processing
- ReportingService: Generating operational reports and analytics
- UserService: User account management

Data Access Layer

The data access layer is responsible for interaction with databases and consists of:

- Repository interfaces that extend Spring Data JPA repositories
- Custom query methods for complex database operations
- Entity classes representing database tables
- Data mapping logic between entities and DTOs

This layer abstracts the underlying database implementation, allowing the service layer to work with domain objects without concerning itself with the details of data persistence.

For complex queries that aren't easily expressed through Spring Data JPA method names, the system uses:

- JPQL (Java Persistence Query Language)
- Native SQL queries with the `@Query` annotation
- Specification pattern for dynamic query building
- 5.4 Authentication & Authorization

The authentication and authorization system is built on Spring Security and includes:

- JWT (JSON Web Token) based authentication
- Role-based access control (RBAC)
- Method-level security using annotations
- Secure password storage with BCrypt hashing

The system supports different user roles such as Pharmacy Managers, Employees, Cashiers, and System Administrators. Each role has specific permissions that determine what operations they can perform.

Authentication flow:

1. User provides credentials
2. System validates credentials against stored user data
3. If valid, a JWT token is generated and returned
4. Subsequent requests include this token for authentication
5. Token expiration and refresh mechanisms ensure security while maintaining usability

5.6 Database Architecture

5.6.1 Schema Design

The database schema is designed to support the complex relationships between entities in the pharmacy management domain. Major tables include:

- users: Stores user account information
- pharmacies: Stores pharmacy location and basic information
- employees: Links users to pharmacies with employment details
- drugs: Master list of drugs in the system

- `pharmacy_drug`: Maps drugs to pharmacies with inventory information
- `shifts`: Defines working shifts for employees
- `receipt`: Stores sales transaction information
- `sales_receipt`: Contains line items for sales
- `drug_order`: Tracks orders for drug inventory replenishment
- `order_item`: Contains line items for orders
- `sales_operation`: Records all drug-related operations (sales, returns, etc.)

The schema employs normalization principles to minimize redundancy while ensuring efficient data retrieval. Foreign key constraints maintain data integrity across related tables.

5.6.2 Entity Relationships

Key relationships in the database include:

- One-to-Many: A user can manage multiple pharmacies
- Many-to-Many: Pharmacies have multiple drugs, and drugs can be in multiple pharmacies
- One-to-Many: A pharmacy can have multiple employees
- One-to-Many: A receipt can have multiple sales receipt items
- One-to-Many: A drug order can have multiple order items

These relationships are carefully designed to reflect the business domain while maintaining data integrity and query performance.

6 Chapter Six: UI/UX Design and System Screens

6.1 Introduction

This chapter presents the user interface and experience design for both the pharmacy management web dashboard and mobile application. The focus was on creating clear, consistent, and user-friendly interfaces tailored to different pharmacy roles. The dashboard emphasizes administrative control and data visibility, while the mobile app supports fast, intuitive actions like scanning drugs and generating invoices. Screens are designed with accessibility, responsiveness, and ease of use in mind, and each section details the purpose, features, and implementation of major UI components.

6.2 USER INTERFACE DESIGN (UI/UX) - Dashboard

6.2.1 Design Principles

The dashboard's UI was designed with the following principles in mind:

- **Consistency:** Maintains unified layout and color theme throughout all pages.
- **Accessibility:** Components and typography are selected for clear visibility and user comfort.

6.2.2 Design Colours

The dashboard uses colors that balance professionalism and clarity:

- **Cyan Shades** (bg-cyan, cyan-100 to cyan-900): Used for backgrounds and main components to evoke calmness and clarity.
- **Emerald Shades** (emerald-400, emerald-600, etc.): Used for buttons, success states, and accents to promote trust and vibrancy.

6.3 SCREENS AND COMPONENTS – DASHBOARD

6.3.1 Login Screen

- **Purpose:** Allow admin or staff to securely access the dashboard.
- **Features:** Email & password input fields with validation, 'Forgot password' link, Login button styled with emerald background.
- **Implementation:** Widgets: Input, Button, Form, Tailwind classes.

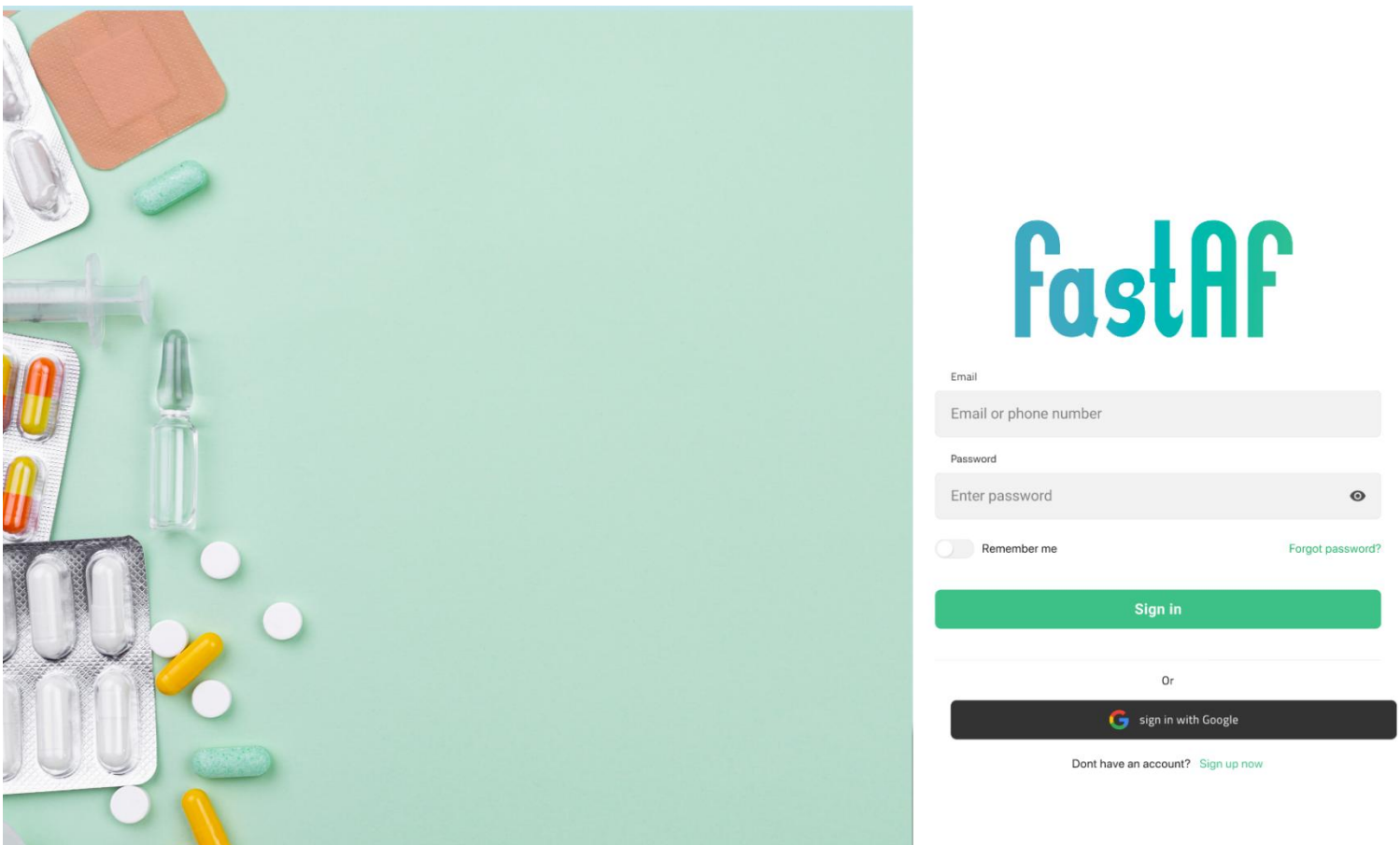
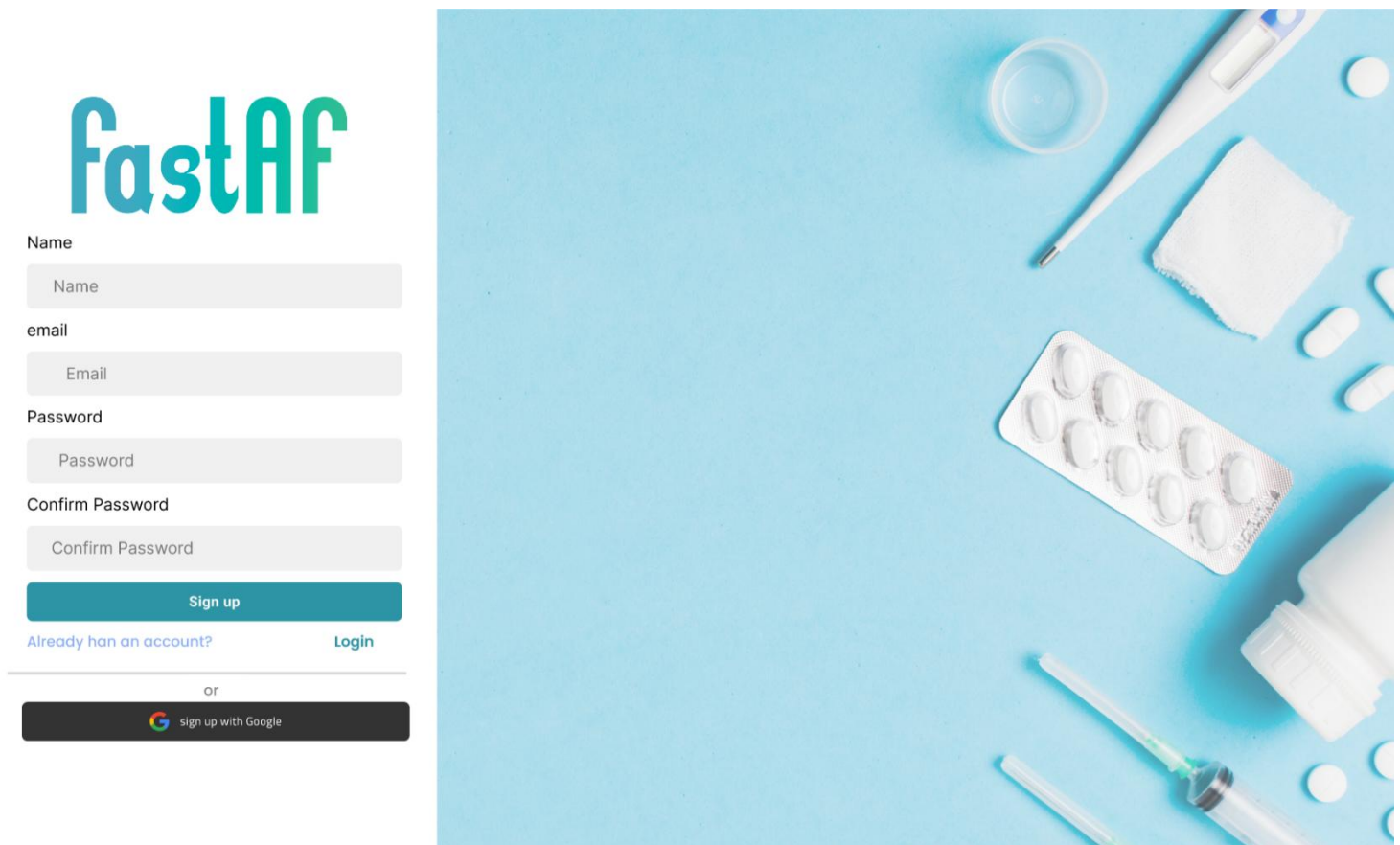


Figure 6-1 login screen

6.3.2 Register Screen

- **Purpose:** Allow pharmacy branches to create admin accounts.
- **Features:** Fields: Full name, email, password, confirm password. Inline validation. Styled call-to-action register button.
- **Implementation:** Widgets: Input, Form, Alert, Validation message.



The image displays the registration interface for 'FastAf' on the left and a medical-themed background on the right. The registration form includes the following elements:

- FastAf** logo in teal and green.
- Name** field: A light gray input box with the placeholder text 'Name'.
- email** field: A light gray input box with the placeholder text 'Email'.
- Password** field: A light gray input box with the placeholder text 'Password'.
- Confirm Password** field: A light gray input box with the placeholder text 'Confirm Password'.
- Sign up** button: A teal button with white text.
- Already han an account?** link: A blue link with a typo.
- Login** link: A blue link.
- or** separator: A small gray text label.
- sign up with Google** button: A dark gray button with the Google logo and white text.

The background image on the right shows various medical supplies on a light blue surface, including a thermometer, a syringe, a pill blister pack, a small container, and several pills.

Figure 6-2 signup screen

6.3.3 Inventory Page

- **Purpose:** Display current stock of medications.
- **Features:** Filters: By category, quantity, expiry. Table layout with pagination.
- **Implementation:** Components: Table, Select, Badge, Search bar.

The screenshot displays the 'Inventory' page of a web application. At the top, there is a navigation bar with a search input field containing the placeholder text 'Type a command or search...'. To the right of the search bar are icons for settings, a share icon, and a user profile. Below the navigation bar, the page title 'Inventory' is prominently displayed. Underneath the title, there is a search input field with the placeholder text 'Search a term..' and a 'Select Filter' dropdown menu. The main content area features a table with the following columns: 'id', 'name', 'form', 'Price ↑↓', 'Expiry Date', and 'stock'. The table contains 12 rows of medication data. At the bottom of the table, there are pagination controls including 'Previous', '1', '2', and 'Next' buttons.

id	name	form	Price ↑↓	Expiry Date	stock
6	Concor 5 mg	Tablet	72 LE	April, 27	7
9	DG wash menthol	Mouth Wash & Gargle	35 LE	April, 27	18
13	Dove nourishing secrets strengthening ritual	Shampoo	130 LE	April, 27	6
16	Garamycine	Topical Cream	22 LE	April, 27	11
23	Limitless omega-3	Capsule	198 LE	April, 27	16
26	One Two Three	Oral Suspension	32 LE	April, 27	18
29	Otrivin 0.1 Adults	Nasal Drops	24 LE	April, 27	11
30	Panadol Advance	Tablet	92 LE	April, 27	5
32	Sensodyne fluoride 100ml	Toothpaste	192 LE	April, 27	17
37	Sensodyne multi care + whitening 50ml	Toothpaste	102 LE	April, 27	14

Figure 6-3 inventory screen

6.3.4 Employees Page

- **Purpose:** Manage pharmacy staff accounts and roles.
- **Features:** Add/remove/edit staff. Role-based filtering (pharmacist, cashier, etc.).
- **Implementation:** Components: Modal, Table, Role filter.

Employees Page

Shifts

night	00:00 - 00:00	:
afterone	05:00 - 09:00	:

Add Shift

Shift Name
e.g. Night Shift

Start Time
Hours Minutes ⌚

End Time
Hours Minutes ⌚

Create Shift

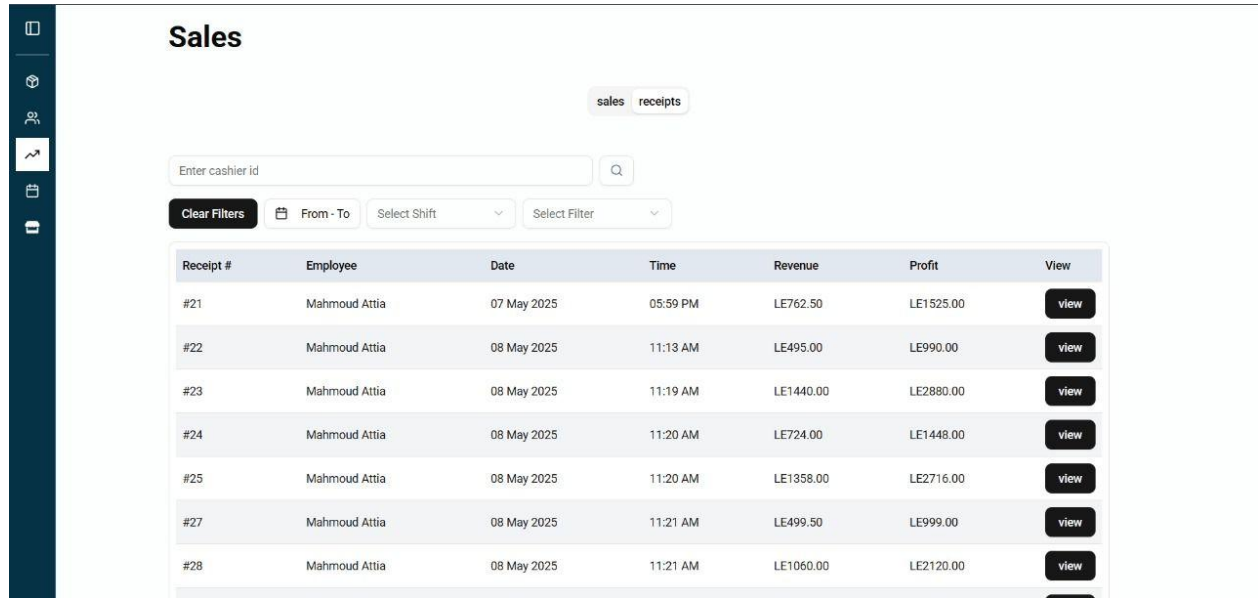
Search a term.. **Create Employee** Select Filter

id	name	email	role	gender	age	shift	
52	Mahmoud Attia	mahmoudattia@gmail.com	manager	male	30	Evening Shift	...
53	Abdulrahman Atef	abdoatef@gmail.com	employee	male	40	Evening Shift	...

Figure 6-4 Employee screen

6.3.5 Sales Page

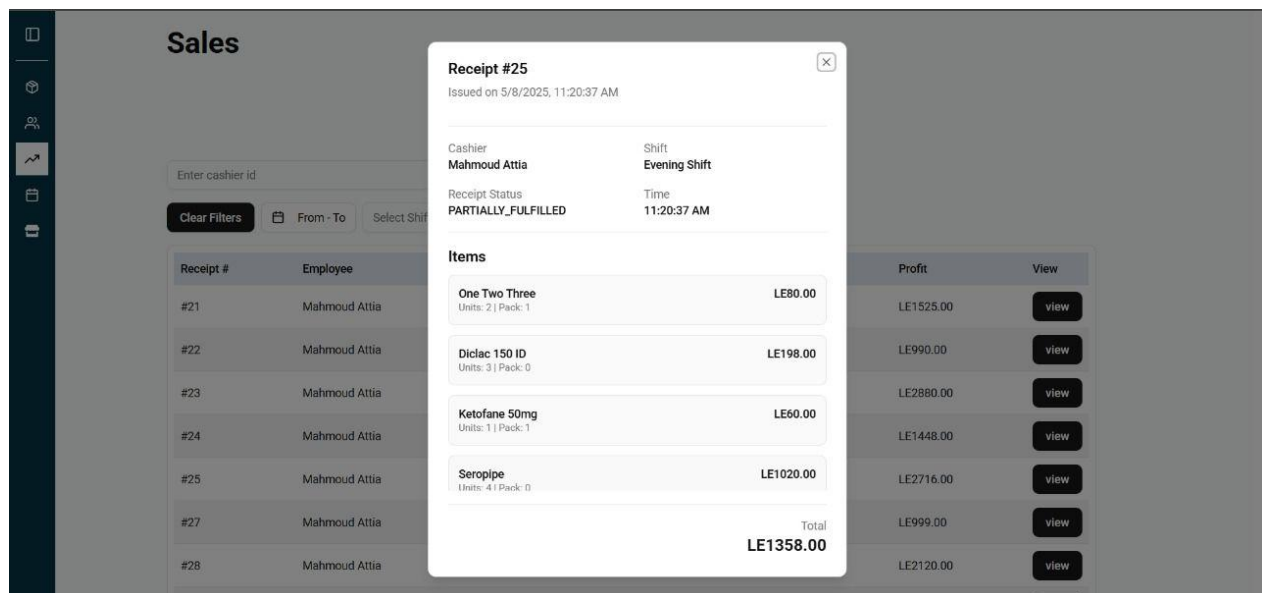
- **Purpose:** view sales and receipts reports.
- **Features:** open a receipt for comprehensive details.
- **Implementation:** Components: Charts, DatePicker, ExportButton.



The screenshot shows the 'Sales' screen with a sidebar on the left containing icons for home, sales, receipts, and a user profile. The main content area has a 'Sales' title, tabs for 'sales' and 'receipts', and a search bar for 'Enter cashier id'. Below the search bar are filters: 'Clear Filters', 'From - To', 'Select Shift', and 'Select Filter'. A table displays sales records with columns: Receipt #, Employee, Date, Time, Revenue, Profit, and View. The table contains 8 rows of data for receipts #21 through #28, all issued by Mahmoud Attia on May 8, 2025.

Receipt #	Employee	Date	Time	Revenue	Profit	View
#21	Mahmoud Attia	07 May 2025	05:59 PM	LE762.50	LE1525.00	view
#22	Mahmoud Attia	08 May 2025	11:13 AM	LE495.00	LE990.00	view
#23	Mahmoud Attia	08 May 2025	11:19 AM	LE1440.00	LE2880.00	view
#24	Mahmoud Attia	08 May 2025	11:20 AM	LE724.00	LE1448.00	view
#25	Mahmoud Attia	08 May 2025	11:20 AM	LE1358.00	LE2716.00	view
#27	Mahmoud Attia	08 May 2025	11:21 AM	LE499.50	LE999.00	view
#28	Mahmoud Attia	08 May 2025	11:21 AM	LE1060.00	LE2120.00	view

Figure 6-5 Sales screen



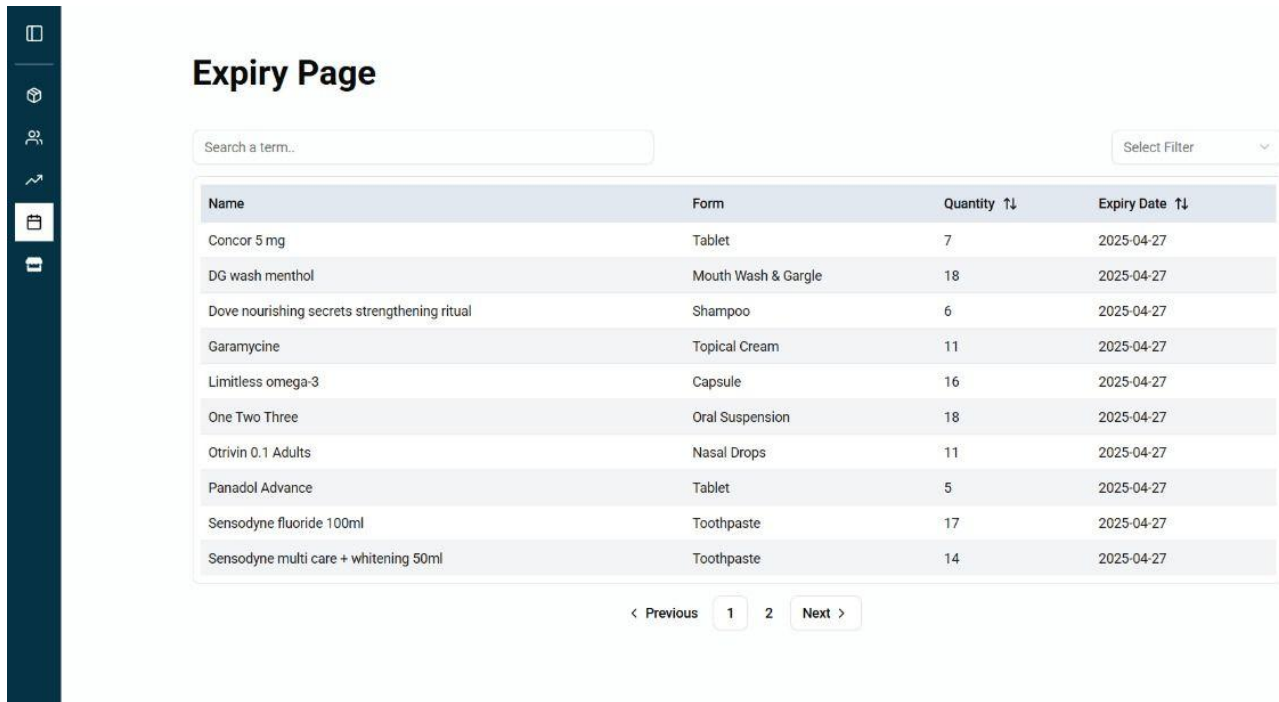
The screenshot shows the 'Receipt #25' modal window. It displays the receipt details: Issued on 5/8/2025, 11:20:37 AM. Cashier: Mahmoud Attia, Shift: Evening Shift. Receipt Status: PARTIALLY_FULFILLED, Time: 11:20:37 AM. The modal lists items with their prices and quantities: One Two Three (LE80.00), Diclac 150 ID (LE198.00), Ketofane 50mg (LE60.00), and Seropipe (LE1020.00). The total is LE1358.00. The background shows the 'Sales' screen with the receipt #25 highlighted in the table.

Receipt #	Employee	Date	Time	Revenue	Profit	View
#21	Mahmoud Attia	07 May 2025	05:59 PM	LE762.50	LE1525.00	view
#22	Mahmoud Attia	08 May 2025	11:13 AM	LE495.00	LE990.00	view
#23	Mahmoud Attia	08 May 2025	11:19 AM	LE1440.00	LE2880.00	view
#24	Mahmoud Attia	08 May 2025	11:20 AM	LE724.00	LE1448.00	view
#25	Mahmoud Attia	08 May 2025	11:20 AM	LE1358.00	LE2716.00	view
#27	Mahmoud Attia	08 May 2025	11:21 AM	LE499.50	LE999.00	view
#28	Mahmoud Attia	08 May 2025	11:21 AM	LE1060.00	LE2120.00	view

Figure 6-6 receipt screen

6.3.6 Expiry Warning Page

- **Purpose:** Track and highlight near-expiry medications.
- **Features:** Auto-highlight entries < Expiry threshold. Filter by category.
- **Implementation:** Components: Table, Badge, Date filter.



Expiry Page

Search a term...

Select Filter

Name	Form	Quantity ↑↓	Expiry Date ↑↓
Concor 5 mg	Tablet	7	2025-04-27
DG wash menthol	Mouth Wash & Gargle	18	2025-04-27
Dove nourishing secrets strengthening ritual	Shampoo	6	2025-04-27
Garamycine	Topical Cream	11	2025-04-27
Limitless omega-3	Capsule	16	2025-04-27
One Two Three	Oral Suspension	18	2025-04-27
Otrivin 0.1 Adults	Nasal Drops	11	2025-04-27
Panadol Advance	Tablet	5	2025-04-27
Sensodyne fluoride 100ml	Toothpaste	17	2025-04-27
Sensodyne multi care + whitening 50ml	Toothpaste	14	2025-04-27

< Previous 1 2 Next >

Figure 6-7 Expiry warning screen

6.3.7 Branches Page

- **Purpose:** Manage different pharmacy branches.
- **Features:** Add/edit branch information. View employees and inventory per branch.
- **Implementation:** Components: Map, Card, Table.

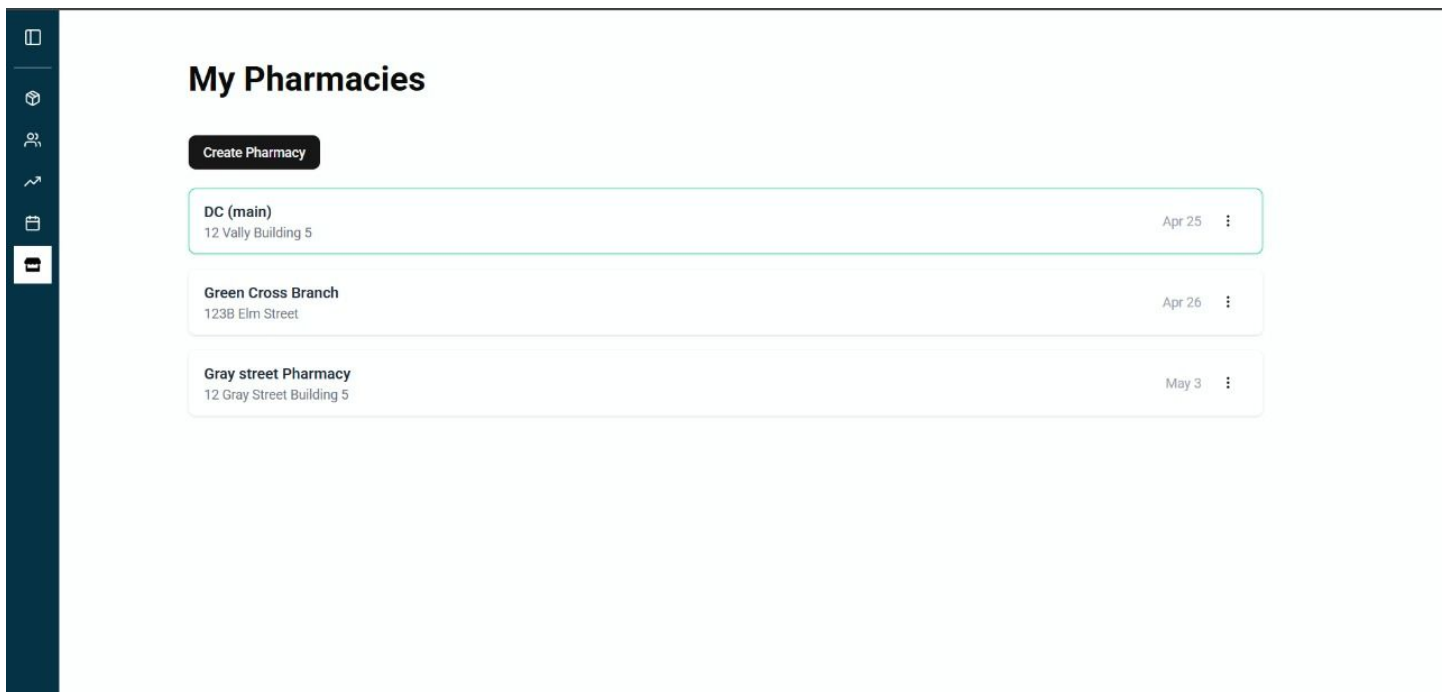


Figure 6-8 branches page

6.4 USER INTERFACE DESIGN (UI/UX) – Application

6.4.1 Design Principles

- **Consistency:**
 - The design ensures all mobile screens follow a unified layout and theming with structured visual hierarchies.
- **Accessibility:**
 - Text sizes, colors, and button placements are optimized for readability and accessibility on small screens.
- **Responsiveness:**
 - UI adjusts dynamically across multiple device sizes using android's responsive packages like Screen Util.

6.4.2 Design Colors

- Dark Blue (#2C3E50):
 - Indicates trust and professional clarity, ideal for the pharmacy context.
- Sky Blue (#3498DB):
 - Used for interaction elements like buttons, reflecting innovation.
- Light Gray (#BDC3C7):
 - Clean background to highlight important UI components.
- White (#FFFFFF):
 - Main background for screens and text, ensuring clarity.
- Emerald (#2ECC71):
 - Used for confirmation elements and success states.

6.5 SCREENS AND WIDGETS – Application

6.5.1 Login Screen

- **Purpose:**
 - Allows pharmacy staff to log into the system securely.
- **Features:**
 - Input fields for email and password with real-time validation.
 - Forgot Password link for password recovery.
 - Login button styled according to system theme.
- **Implementation Details:**
 - Widgets Used: TextFormField, ElevatedButton, GestureDetector.
 - Validation: Implemented using TextEditingController and Form.



Figure 6-9 app login screen

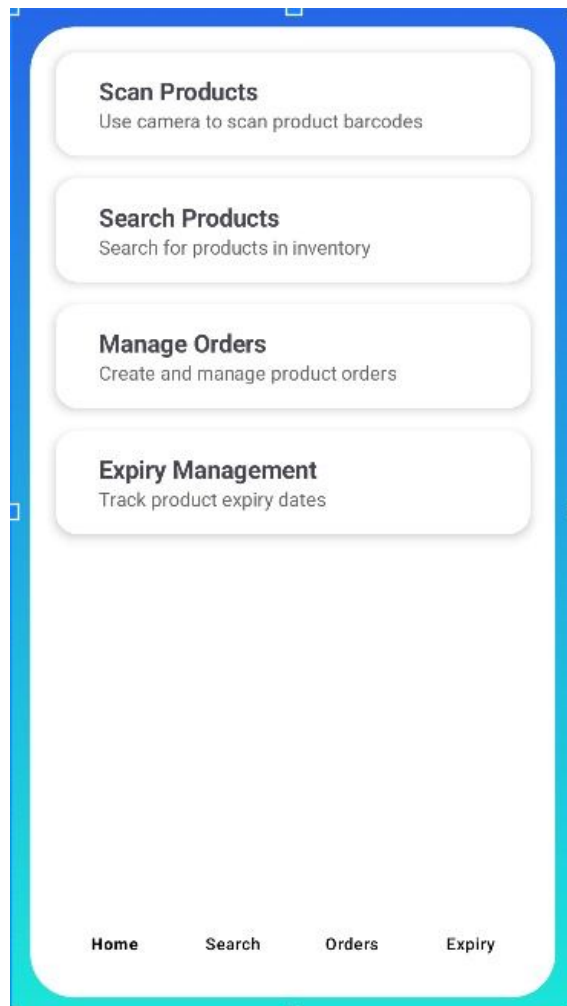


Figure 6-10 pages navigator

6.5.2 Camera Scanner Screen

- **Purpose:**
 - Enables staff to detect a medicine to apply it to the cart or ask for details.
- **Features:**
 - Only camera access allowed; gallery access is disabled.
 - Displays visual feedback post object detected.
 - Camera interface overlays guidance for better capture.
- **Implementation Details:**
 - Widgets Used: CameraController, Stack, ElevatedButton.
 - Logic: Uses camera package for image stream + ML model backend.

6.5.3 Medicine Details Screen

- **Purpose:**
 - Shows detailed information for selected medicine.
- **Features:**
 - Medicine photo, name, expiry date, usage instructions.
 - Filters by category or stock status.
- **Implementation Details:**
 - Widgets Used: Card, Container, ListView.
 - Filtering handled via dropdown filters with real-time updates.

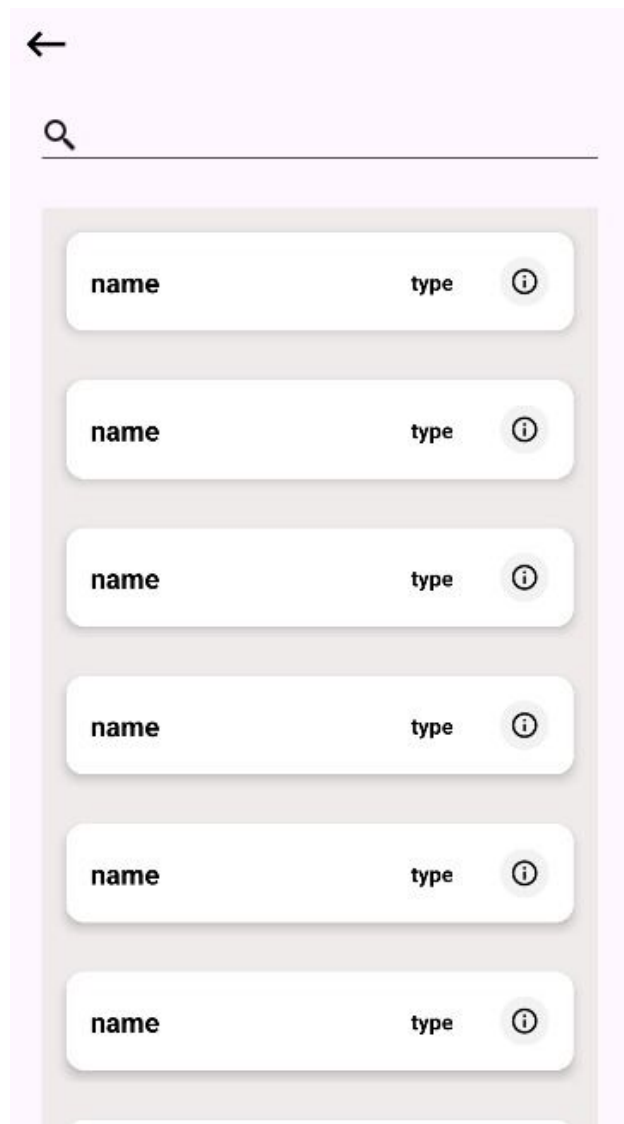


Figure 6-11 search for drug info

6.5.4 Invoice Screen

- **Purpose:**
 - Generates and displays pharmacy sales invoices.
- **Features:**
 - Searchable list of medicines added to invoice.
 - Auto-calculate totals, VAT, discounts.
 - Export options: PDF or print.
- **Implementation Details:**
 - Widgets Used: ListView.builder, DataTable, Print Plugin.
 - State Managed with Cubit/Bloc for form reactivity.

The screenshot displays a mobile application interface for a pharmacy cart. At the top left is a back arrow. Below it are two blue buttons: 'CAM' with a camera icon and 'Search'. The main area contains three identical white rounded rectangular cards, each representing a cart item. Each card has the text 'panadol tablet' at the top. Below this, there are two input fields: 'unit' and 'packs'. To the right of these fields is a 'stock' label. At the bottom right of each card is a quantity '1'. At the bottom of the screen is a large blue 'Submit' button.

Figure 6-12 cart screen

ENTER UR ORDER NAME

name type	amount
name type	amount
name type	amount
name type	amount
name type	amount

Confirm

Figure 6-13 confirm order

Drug Name	
Form	
Shortage	0
Drug Name	
Form	
Shortage	0
Drug Name	
Form	
Shortage	0
Drug Name	
Form	
Shortage	0
Drug Name	
Form	
Shortage	0

Send to Preview

Figure 6-14 make order

←

🔍

Not Expired Approaching Expiry Expired

Name form stock daye
Name form stock daye
Name form stock daye
Name form stock daye

Figure 6-15expiry warning page

7 Chapter Seven: Backend services

7.1 Database Optimization

Several optimization techniques are employed to ensure database performance:

- **Indexing:** Appropriate indexes on frequently queried columns and join columns
- **Materialized Views:** For complex aggregation queries used in reporting
- **Partitioning:** For large tables with time-based data (like transactions)
- **Regular VACUUM and ANALYZE:** To maintain PostgreSQL's query optimizer effectiveness
- **Proper Data Types:** Using appropriate data types to minimize storage requirements

Additionally, database-specific optimizations include:

- Using JSONB for semi-structured data when appropriate
- Leveraging PostgreSQL's array types for certain use cases
- Employing covering indexes for frequently executed queries

7.2 Search Functionality

The system implements robust search capabilities using PostgreSQL's full-text search features:

- **tsvector columns:** For efficient text searching
- **GIN indexes:** To speed up full-text search operations
- **pg_trgm extension:** For fuzzy matching and similarity searches
- **Search vector triggers:** To automatically update search vectors when data changes

This approach provides powerful search capabilities for drug names, pharmacy information, and other text-heavy data.

7.3 API Design

7.3.1 RESTful Principles

The API adheres to RESTful principles for a consistent and intuitive interface:

- **Resource-Based:** Endpoints are organized around resources (pharmacies, drugs, users)
- **HTTP Methods:** Proper use of GET, POST, PUT, PATCH, and DELETE methods
- **Statelessness:** No client state is stored on the server between requests
- **Uniform Interface:** Consistent patterns for resource identification and manipulation
- **HATEOAS:** Where appropriate, responses include links to related resources

These principles make the API intuitive to use and maintain, following widely accepted industry standards.

7.3.2 API Endpoints Structure

The API endpoints follow a hierarchical structure that reflects the relationships between resources:

- `/api/v1/pharmacies`: Pharmacy management
- `/api/v1/pharmacies/{id}/drugs`: Drugs in a specific pharmacy
- `/api/v1/pharmacies/{id}/employees`: Employees in a specific pharmacy
- `/api/v1/receipts`: Sales transactions
- `/api/v1/orders`: Drug ordering
- `/api/v1/users`: User management

This structure makes the API intuitive and self-documenting, allowing consumers to understand the relationship between resources from the URL patterns.

7.3.3 Request/Response Format

All API communication uses JSON format with consistent structures:

- **Requests:** Use DTOs with field validation annotations

- **Responses:** Include appropriate HTTP status codes and structured data
- **Pagination:** pagination for list endpoints using page, size, and sort parameters
- **Filtering:** Query parameters for filtering collections
- **Sorting:** Parameters to specify sort fields and direction

This consistency makes the API easier to consume and integrate with other systems.

7.3.4 Error Handling

The API implements comprehensive error handling:

- **HTTP Status Codes:** Appropriate status codes for different error scenarios
- **Error Responses:** Structured JSON responses with error codes, messages, and details
- **Validation Errors:** Detailed information about validation failures
- **Exception Handling:** Global exception handlers to ensure consistent error responses

This approach provides clear feedback to API consumers when issues occur, improving the developer experience and making troubleshooting more efficient.

7.4 Caching Implementation

7.4.1 Redis Architecture

The Redis caching layer is designed to optimize performance for frequently accessed data:

- **Single Redis Instance:** For development and testing environments
- **Redis Cluster:** For production deployment with high availability
- **Dedicated Redis Database:** Separate from any other applications
- **Connection Pooling:** To efficiently manage Redis connections

Redis is configured with appropriate memory limits and eviction policies to ensure optimal performance even under heavy load.

7.4.2 Caching Strategy

The caching strategy is tailored to the specific needs of the pharmacy system:

- **User Authentication Data:** Caching user sessions and authentication tokens
- **Reference Data:** Caching relatively static data like drug information
- **Inventory Information:** Caching frequently accessed inventory status
- **Pharmacy Details:** Caching basic pharmacy information

Each cache entry has an appropriate Time-To-Live (TTL) value based on the volatility of the data and the tolerance for potential staleness.

Cache keys follow a consistent naming convention: `entity:id:attribute` or `entity:query:parameters` to make debugging and management easier.

7.4.3 Cache Invalidation

To maintain data consistency, the system implements strategic cache invalidation:

- **Write-Through:** Updating both the database and cache when data changes
- **Event-Based Invalidation:** Cache entries are invalidated based on specific application events
- **TTL-Based Expiration:** All cached items have a maximum lifetime
- **Manual Purge:** Administrative capabilities to clear specific cache entries when needed

These strategies ensure that cached data remains reasonably fresh while still providing performance benefits.

7.5 Security Architecture

7.5.1 Authentication Flow

The authentication process follows industry best practices:

1. User submits credentials (username/password)
2. System validates credentials against the database (with passwords stored as BCrypt hashes)
3. Upon successful validation, a JWT token is generated with user identity and roles
4. The token is returned to the client for use in subsequent requests

5. Each API request includes the token in the Authorization header
6. The system validates the token and extracts user identity and permissions
7. Access is granted or denied based on the user's roles and the requested operation

The JWT token has a limited lifetime, and a refresh token mechanism allows extending the session without requiring the user to re-authenticate frequently.

7.5.2 Authorization Model

Authorization is implemented using a role-based approach:

- **System Administrator:** Full access to all system functions
- **Pharmacy Manager:** Management of their pharmacy, inventory, and staff
- **Pharmacist:** Access to drug information and inventory operations
- **Cashier:** Sales transactions and receipt operations
- **Employee:** Basic access to their pharmacy's information

Permissions are checked at multiple levels:

- API endpoint access (URL-based authorization)
- Method-level security (using Spring Security annotations)
- Data-level security (ensuring users can only access data they are authorized to see)

This multi-layered approach ensures robust access control throughout the application.

7.5.3 Data Protection

Sensitive data is protected through various mechanisms:

- **Password Storage:** BCrypt hashing for passwords
- **Data Encryption:** Encryption for sensitive data at rest
- **HTTPS:** All API communication uses TLS/SSL encryption
- **Data Masking:** Sensitive information is masked in logs and error reports
- **Input Validation:** Strict validation to prevent injection attacks
- **Output Encoding:** Proper encoding to prevent XSS vulnerabilities

These measures form a comprehensive approach to data protection throughout the system.

7.5.4 API Security

API security is enforced through:

- **JWT Authentication:** For identity verification
- **Rate Limiting:** To prevent abuse and DoS attacks
- **CORS Configuration:** To control which domains can access the API
- **Input Validation:** To prevent injection attacks and invalid data
- **Request Logging:** For audit trails and security monitoring

These measures protect the API from common vulnerabilities and attacks while maintaining usability for legitimate clients.

7.6 Deployment Architecture

7.6.1 CI/CD Pipeline

The continuous integration and delivery pipeline is implemented using GitHub Actions:

1. **Code Push Trigger:** Pipeline starts automatically when code is pushed to specific branches
2. **Build:** The application is built using Maven
3. **Unit Tests:** Automated tests are run to verify code functionality
4. **Docker Image Build:** Creation of Docker container images
5. **Image Registry Push:** Images are pushed to Azure Container Registry
6. **Deployment:** Automatic deployment to development environments, manual approval for production

This pipeline ensures consistent, reliable deployments with proper validation at each stage.

7.6.2 Infrastructure Setup

The infrastructure evolved through two main phases:

Phase 1: Azure VM Deployment

- Ubuntu Server VM on Azure
- PostgreSQL database as a service
- Redis Cache as a service
- Application running directly on the VM with systemd service management

Phase 2: Azure Web App with Containers

- Azure Web App service for hosting
- Container-based deployment
- Managed PostgreSQL database
- Azure Redis Cache
- Azure Application Insights for monitoring

The migration from VM to Web App provided better scalability, easier management, and improved integration with Azure's monitoring and security services.

7.6.3 Containerization

The application is containerized using Docker with a multi-stage build approach:

1. **Build Stage:** Uses Maven to compile and package the application
2. **Runtime Stage:** Uses a minimal JRE image for the application runtime

The Docker image includes:

- The application JAR file
- Runtime configuration
- Health check definitions
- Proper user permissions (non-root execution)
- Optimized JVM settings

Containerization provides consistent environments across development, testing, and production, minimizing "works on my machine" issues and simplifying deployment.

7.6.4 Cloud Resources

The Azure resources utilized include:

- **Azure Web App:** For hosting the containerized application
- **Azure Database for PostgreSQL:** Managed PostgreSQL service
- **Azure Cache for Redis:** Managed Redis service
- **Azure Storage Blob:** Storage Image files as Blobs
- **Azure Container Registry:** For storing Docker images
- **Azure Application Insights:** For application monitoring
- **Azure Key Vault:** For secure credential storage
- **Azure Virtual Network:** For network isolation and security

These managed services reduce operational overhead and provide scalability, high availability, and security features out of the box.

7.7 Scalability Considerations

7.7.1 Horizontal Scaling

The application is designed to support horizontal scaling:

- **Stateless Design:** No session state stored in the application
- **Centralized Caching:** Redis as a shared cache across instances
- **Connection Pooling:** Efficient management of database connections
- **Containerization:** Easy deployment of multiple instances

In the Azure Web App environment, scaling is managed through:

- **Auto-scaling Rules:** Based on CPU, memory, and request queue metrics
- **Scale-out Limits:** Configured according to anticipated load
- **Geographic Distribution:** Using Traffic Manager for global distribution (future enhancement)
-

7.7.2 Database Scaling

The database scaling strategy includes:

- **Read Replicas:** For distributing read operations
- **Connection Pooling:** To efficiently manage database connections
- **Query Optimization:** Regular performance tuning of critical queries
- **Indexing Strategy:** Continuously refined based on query patterns
- **Data Partitioning:** For large tables with clear partitioning keys

As the system grows, consideration will be given to implementing database sharding for further horizontal scaling.

7.7.3 Cache Scaling

The Redis caching layer is designed to scale with the application:

- **Redis Cluster:** For distributed caching with high availability
- **Memory Optimization:** Careful selection of data structures and TTL values
- **Monitoring:** Cache hit rates and memory usage to identify scaling needs

The caching strategy is regularly reviewed to ensure it continues to provide performance benefits as data volumes and access patterns evolve.

7.8 Monitoring & Observability

7.8.1 Logging Strategy

The logging framework provides comprehensive visibility into system operation:

- **Structured Logging:** JSON-formatted logs for easier parsing and analysis
- **Log Levels:** Appropriate use of DEBUG, INFO, WARN, and ERROR levels
- **Contextual Information:** Including user ID, request ID, and other relevant context
- **Sensitive Data Masking:** Ensuring no sensitive information is logged
- **Centralized Log Collection:** Using Azure Log Analytics

The logging strategy balances between providing sufficient information for troubleshooting and avoiding excessive log volume that could impact performance.

7.8.2 Performance Monitoring

Performance monitoring is implemented using Azure Application Insights:

- **Request Timing:** Tracking response times for all API endpoints
- **Database Performance:** Monitoring query execution times
- **Cache Effectiveness:** Tracking cache hit/miss ratios
- **Resource Utilization:** CPU, memory, and network usage
- **Custom Metrics:** Application-specific performance indicators

Dashboards provide real-time visibility into system performance, helping to identify bottlenecks and optimization opportunities.

7.8.3 Alert System

The alerting system ensures timely response to operational issues:

- **Threshold-Based Alerts:** For metrics exceeding defined thresholds
- **Anomaly Detection:** Identifying unusual patterns in metrics
- **Availability Monitoring:** Alerts for service outages
- **Error Rate Monitoring:** Notifications for increased error rates
- **Business Metric Alerts:** Monitoring key business indicators

Alerts are routed to appropriate channels (email, SMS, Teams) based on severity and time of day, ensuring the right people are notified when issues occur.

7.9 Testing Strategy

7.9.1 Unit Testing

The unit testing approach focuses on thorough coverage of business logic:

- **Service Layer Testing:** Comprehensive tests for business logic
- **Mocking:** Using Mockito to isolate components for testing
- **Parameterized Tests:** For testing multiple scenarios efficiently

- **Test Data Builders:** To simplify test setup and improve readability
- **Coverage Targets:** Minimum 80% code coverage for critical components

Unit tests are run automatically in the CI pipeline, preventing the introduction of regressions.

7.9.2 Integration Testing

Integration tests verify the interaction between components:

- **Repository Testing:** Testing database interactions with an in-memory database
- **API Testing:** Validating API endpoints with Spring MVC Test
- **Security Testing:** Verifying authentication and authorization rules
- **Transaction Management:** Ensuring proper transaction behavior

These tests catch issues that might not be apparent in unit tests, particularly around component integration and configuration.

7.9.3 Performance Testing

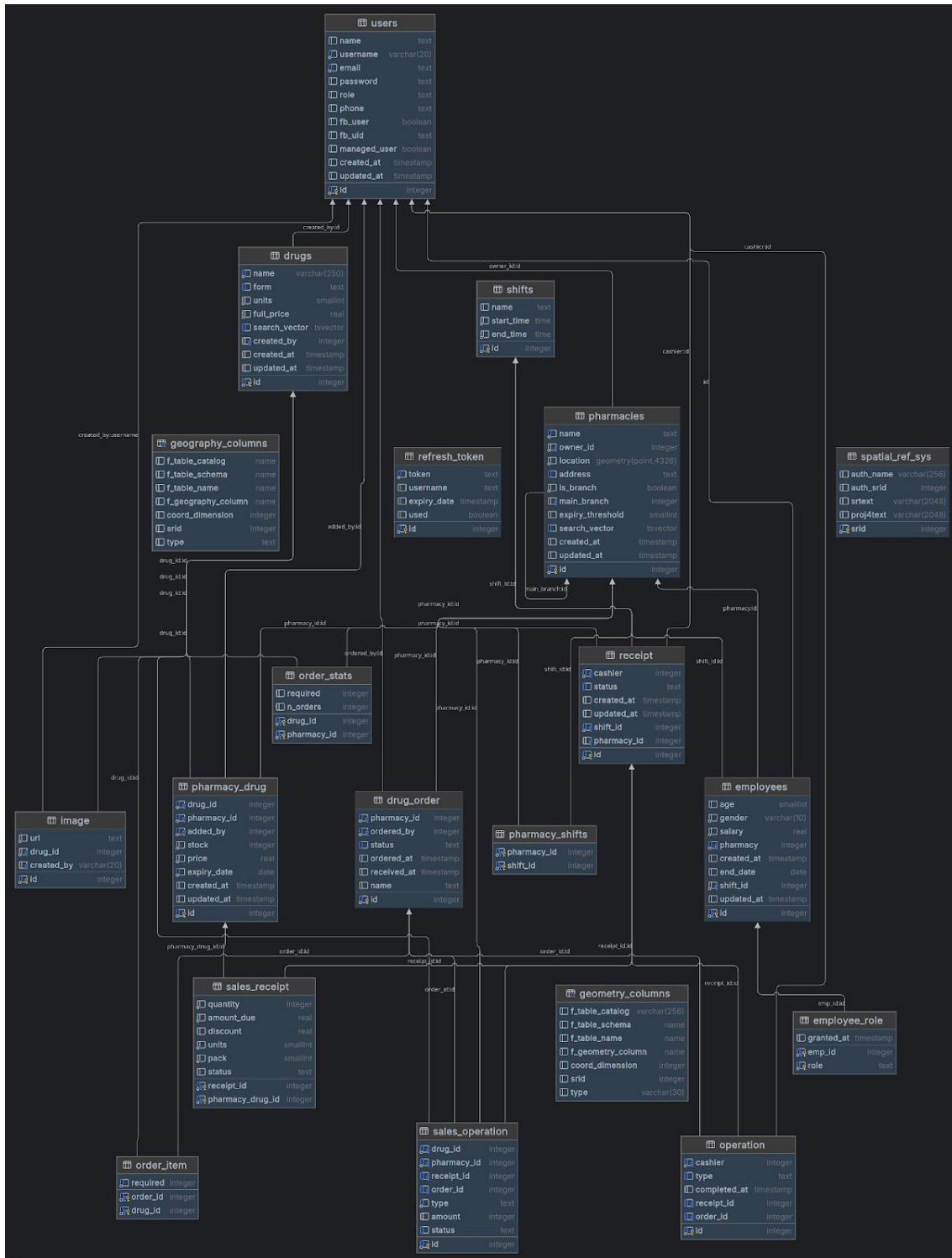
Performance testing ensures the system meets performance requirements:

- **Load Testing:** Simulating expected user loads to verify response times
- **Stress Testing:** Identifying breaking points by gradually increasing load
- **Endurance Testing:** Verifying system stability under sustained load
- **Database Performance:** Testing query performance with realistic data volumes

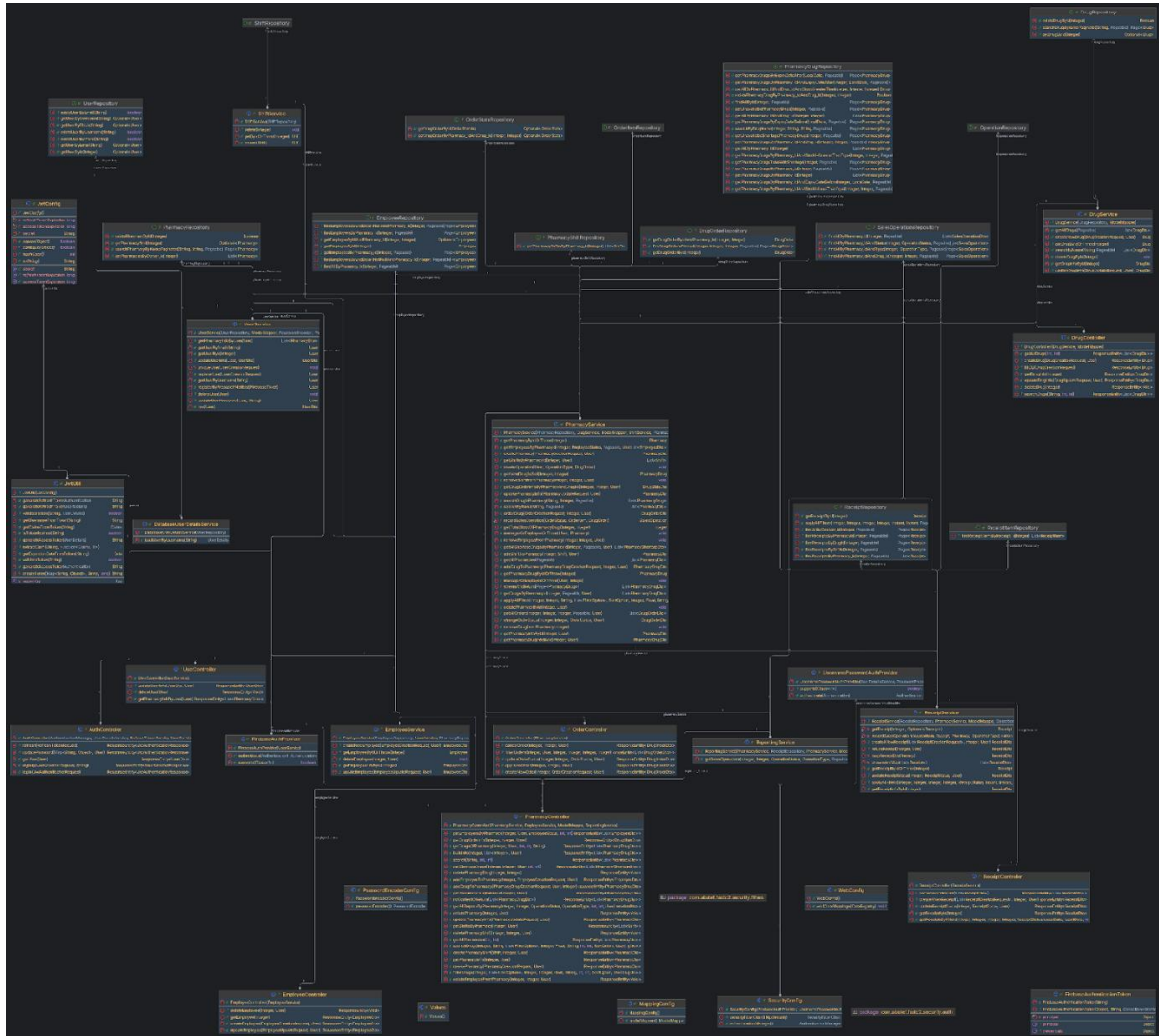
Performance testing is performed in environments that closely mirror production, using tools like Apache JMeter and Azure Load Testing.

7.10 Appendices

7.10.1 Database Schema Diagram



7.10.2 Architecture Diagrams



7.10.3 API Documentation Reference

The complete API documentation is available through Swagger/OpenAPI at </swagger-ui.html> when the application is running. This provides an interactive interface for exploring and testing the API endpoints.

For offline reference, the OpenAPI specification is also available as a JSON or YAML file in the project repository.

8 Chapter Eight: FrontEnd and Application implementation

8.1 introduction

This document provides detailed documentation for the Home page components of the FastAF project. The Home page contains several key sections that are implemented as separate React components. These components work together to form the complete layout of the Home page, using Tailwind CSS for styling and React for functionality.

8.2 Main Page Component

The Home component is the main landing page of the FastAF pharmacy management application. It brings together all major sections of the homepage using modular React components.

Imported Components:

Table 8-1front components

Component	Description
Header :	Fixed navigation bar that changes style on scroll
Hero :	Hero section with a call-to-action and headline
StackedFeaturesSection :	Features section with scroll-based animated cards
TestimonialSection :	Carousel of user testimonials to build credibility
FrequentlyAsked :	Accordion-style FAQ section

Component	Description
Footer:	Bottom part of the page with navigation and legal links

The Header component is responsible for displaying the navigation bar at the top of the page. It changes its appearance when the user scrolls down, becoming transparent and adding a shadow effect.

Features:

- Displays blogo and navigation links Changes style on scroll
- Sign up and login buttons

Technologies Used:

Table 8-2 tools used

Tool / Library	Purpose
React:	Component-based UI
Framer Motion:	Animation and scroll detection
Tailwind CSS:	Utility-first styling
React Router DOM:	Page navigation
React Icons:	Hamburger icon (planned)
Custom Utility:	<code>cn ()</code> for conditional classe

The Hero component displays the main heading and introductory text, along with a call-to-action button. It features a background animation to give it a dynamic feel.

- **Background Design:** The `BackgroundShapes` component creates an engaging visual background that enhances the user experience.
- **Main Heading:** A bold headline emphasizing the core message ("Pharmacy Management Easier Than Before").
- **Subheading:** A description of the app's functionality ("FastAF simplifies pharmacy management...").
- **CTA Button:** A "Try now for free" button that redirects users to the sign-up page.
- **Visual Effects:** Includes hover effects on the CTA button and a bouncing animation for added interactivity.

Table 8-3front libraries used

Tool / Library	Purpose
React:	Component-based architecture.
React Router DOM:	Navigation to the sign-up page (<code>useNavigate</code>).
React Icons:	For the right arrow icon in the button (<code>FaAngleRight</code>).
Tailwind CSS:	For styling the layout and animations.

Technologies Used:

Table 8-4front technologies used

Element	Description
Main Heading	Large, bold text to capture attention ("Pharmacy Management Easier Than Before").
Subheading	Provides a brief overview of what the application does, emphasizing simplicity.
CTA Button	Upon clicking, it navigates the user to the sign-up page (<code>/sign-portal</code>). The button also features hover effects and an animated background.
Bouncing Indicator	A small animated circular shape that draws attention to the CTA section.

Additional_Notes:

- The `BackgroundShapes` component is used to create a visually interesting background that complements the overall design.
- The button has interactive hover effects, where the background gradient changes, and the button itself becomes slightly more pronounced.

- The layout is mobile-responsive, and the typography adjusts accordingly for different screen sizes.
- The `FaAngleRight` icon from React Icons is used as a visual cue to indicate the user will move forward by clicking the button.

The `StackedFeaturesSection` component is designed to showcase a series of feature cards in a stacked, scrollable layout. This section utilizes the `motion` library to create scroll-based animations, enhancing the user experience as the cards are revealed based on the user's scroll position. Each feature card contains information from a predefined data set (`featureCards`).

Key Features:

- **Scroll-Based Animations:** Each feature card fades in or adjusts its position based on the scroll progress, creating an engaging experience as the user scrolls through the section.
- **Dynamic Feature Cards:** The feature cards are dynamically generated from an array (`featureCards`), making the section highly reusable and easy to modify.
- **Responsive Layout:** The component is designed to work across different screen sizes with a full-height section (`min-h-screen`).

Table 8-5 front libraries

Tool / Library	Purpose
React	Component-based architecture.
Motion React	For scroll-based animations using <code>useScroll</code> .
Tailwind CSS	For styling and responsive layout.

Table 8-6 front tools

Element	Description
<code>scrollYProgress</code>	Tracks the scroll position of the container and determines when the feature cards will appear.
Feature Cards	Cards are rendered dynamically using data from the <code>featureCards</code> array. They respond to scroll events to animate their appearance.

Element	Description
StackedFeatureCard	A reusable component that receives props such as the scroll progress, feature data, and index to render the individual cards with animations.

Additional Notes:

- The `useRef` hook is used to track the container element, allowing for scroll-based animations.
- The `useScroll` hook from `motion/react` is used to track the user's scroll position relative to the container div.
- The `featureCards` array is a constant that contains the data for the features to be displayed. This makes it easy to modify or add new features without changing the component logic.
- Each feature card animates based on the scroll progress, making the section interactive and visually appealing as the user scrolls.

8.3 StackedFeatureCard

The `StackedFeatureCard` component is designed to render individual feature cards within the `StackedFeaturesSection`. Each card represents a unique feature with its own title, description, and a list of benefits. The component uses scroll animations to scale each card based on the user's scroll progress, creating an engaging visual effect. The card content is dynamically passed in via props, allowing for flexibility and reusability.

Key

Features:

- **Dynamic Scaling:** The cards scale dynamically as the user scrolls through the section, with each card's size decreasing as it moves down the screen.
- **Sticky Positioning:** The cards are positioned in a sticky layout, ensuring they stay in view as the user scrolls.

- **Customizable Content:** Each feature card's content (title, description, benefits, image) is passed as props, making the component reusable for different features.
- **Animated Transitions:** The scaling effect is driven by the scroll progress, providing a smooth animation as the user interacts with the page.
- **Gradient Background:** The first card displays a gradient background for added visual interest.

Technologies Used:

Tool / Library	Purpose
React:	Component-based architecture.
Motion React:	Used for scroll animations and transitions (<code>motion.div</code> , <code>useTransform</code>).
Tailwind CSS:	For styling the layout and animations.

Behavior Description:

Table 8-7 Behavior Description

Element	Description
Dynamic Scaling	The card's scale adjusts based on the scroll progress (<code>useTransform</code>), making each card smaller as the user scrolls.
Sticky Positioning	The cards stay sticky in the viewport as the user scrolls down, allowing for an interactive, fixed card layout.
Gradient Background	The first feature card displays a gradient background to enhance visual appeal.
Content	The component dynamically renders the feature content (title, description, benefits, image) from props passed down from the parent component.

Additional Notes:

- The `useTransform` hook is used to calculate the scaling factor for each card based on the scroll progress.

- The card includes a list of benefits, each of which is rendered with a checkmark icon for better readability and visual appeal.
- The `StackedFeatureCard` is a reusable component, making it easy to render different features by simply passing different props.
- The first card has an additional gradient background (created via Tailwind classes) for added visual interest.
- The component is responsive, with the image only visible on medium screens (`md:flex`).

8.4 TestimonialSection

The `TestimonialSection` component is responsible for displaying a section of testimonials from customers, showcasing their trust in the service. It features an auto-scrolling carousel of testimonial items that is looped continuously. The section includes a heading and a brief description at the top, explaining the value of FastAF's pharmacy management solution. The component uses the `motion` library to animate the carousel of testimonials.

Key Features:

- **Auto-Scrolling Carousel:** The testimonials are displayed in a continuously looping carousel that moves horizontally.
- **Customer Testimonials:** Displays multiple testimonials, with each one rendered via the `TestimonialList` component.
- **Smooth Animation:** Utilizes `motion.div` to animate the carousel's horizontal scrolling effect, creating a smooth, seamless transition between testimonials.
- **Responsive Design:** The component is built with responsive styles, ensuring it looks good on all screen sizes.
- **Gradient Button:** Includes a `GradientButton` that animates into view, adding a stylish call to action.

Technologies Used:

Tool / Library	Purpose
React	Component-based architecture.
Motion React	Used for smooth animations (carousel scrolling, button fade-in).

Tool / Library	Purpose
Tailwind CSS	For styling and layout.

Behavior Description:

Table 8-8 Behavior Description

Element	Description
Auto-Scrolling Carousel	The <code>motion.div</code> element animates the horizontal scroll of testimonials using the <code>animate</code> prop to translate the x-axis over a period of time, creating a smooth scrolling effect.
TestimonialList	A child component responsible for rendering the list of individual testimonials. It is reused in the carousel to show the same set of testimonials twice, ensuring a continuous loop.
Gradient Button	A visually striking button that animates in with a gradient effect, calling attention to the "Seamless experience" message. The button is animated using <code>motion.div</code> with an opacity fade-in.
Responsiveness	The component's layout is designed to be responsive, with the carousel wrapping properly on smaller screens. Tailwind classes are used for fluid scaling across devices.

Additional Notes:

- **Carousel Animation:** The carousel's scrolling effect is infinite (`repeat: Infinity`), creating an endlessly looping animation.
- **Smooth Transitions:** `motion.div` is used to animate the carousel with a smooth transition, making it feel seamless and natural. The duration and easing ensure the scroll is smooth and continuous.
- **Gradient Button Animation:** The `GradientButton` component animates with a delayed fade-in effect when it comes into the view of the user. This is controlled by the `whileInView` and `initial` properties in the `motion.div`.

8.5 6. FrequentlyAsked

The `FrequentlyAsked` component is designed to display a list of frequently asked questions (FAQs) using an accordion-style interface. The component uses the `Accordion` component from the UI library to provide a collapsible and expandable view for each question. It aims to provide helpful information to users, ensuring that common queries are addressed efficiently.

Key Features:

- **Accordion Layout:** Utilizes the `Accordion` component to allow users to toggle the visibility of answers to frequently asked questions.
- **Collapsible Items:** Each FAQ item (question and answer) is contained within an `AccordionItem`. The content remains hidden until the user clicks to reveal it.
- **Responsive Design:** Tailwind CSS is used for styling, ensuring the layout is responsive and adapts to different screen sizes.

Technologies Used:

Table 8-9 Technologies used

Tool / Library	Purpose
React	Component-based architecture.
Tailwind CSS	For styling the layout and responsiveness.
UI Components	Uses custom accordion UI components for better interaction (<code>Accordion</code> , <code>AccordionItem</code> , etc.).

Behavior Description:

Table 8-10 behavior descriptions

Element	Description
Accordion	This component uses an accordion structure that allows only one item to be expanded at a time (<code>type="single"</code>) while also allowing it to collapse (<code>collapsible</code> attribute).
AccordionItems	Each <code>AccordionItem</code> represents an individual FAQ. It consists of a <code>AccordionTrigger</code> (question) and <code>AccordionContent</code> (answer).
Tailwind CSS Classes	The layout is styled with utility-first classes from Tailwind, such as <code>grid</code> , <code>gap-10</code> , <code>text-cyan-700</code> , and <code>border-2</code> for spacing, colors, and borders.
Typography	The questions use bold text (<code>text-4xl font-bold text-cyan-700</code>), while the answers use a more subtle text style (<code>text-base text-gray-600</code>).

Additional Notes:

- **Accordion Structure:** The `Accordion` is configured to only allow a single item to be opened at a time. Each `AccordionItem` represents a single FAQ and contains the question as a trigger and the answer as the content.
- **Custom Styling:** The component leverages custom classes to ensure the accordion is styled appropriately for a clean and modern UI.
- **Component Reusability:** By using a reusable `Accordion` structure, this component can easily be extended with additional FAQ items in the future

7.Footer

The `Footer` component is designed to provide essential information and links at the bottom of the page. It includes copyright information, navigation links for signing up/signing in, and links for privacy policy and terms of service.

Key Features:

- **Copyright Notice:** Displays the copyright for FastAF and mentions all rights are reserved for 2025.
- **Sign Up/Sign In Links:** Provides links to sign-up and sign-in pages.
- **Privacy & Terms Links:** Displays text links for "Privacy Policy" and "Terms of Service" (though the links for these are not yet specified in the code).

Technologies Used:

Table 8-11 technologies used

Tool / Library	Purpose
React	Component-based architecture.
React Router	Used for navigation with the <code>Link</code> component.
Tailwind CSS	For styling the layout and spacing.

Behavior Description:

Table 8-12 behavior description

Element	Description
Copyright Text	Displays "© 2025 FastAf. All rights reserved" in a smaller font size (<code>text-sm</code>).
Links Section	Contains navigation links for "Sign up" and "Sign in", as well as text links for "Privacy Policy" and "Terms of Service".
Tailwind CSS	The component uses utility classes like <code>flex</code> , <code>space-x-2</code> , <code>divide-x</code> , <code>py-3</code> , and <code>text-sm</code> to ensure proper alignment, spacing, and styling.

Additional Notes:

- **Navigation:** The `Link` components from React Router are used to enable client-side routing. The links for signing up and signing in currently point to the same page (`/sign-up`), but these can be updated to point to distinct pages if needed.

- **Styling:** Tailwind CSS provides flexibility for layout and spacing. The links are divided by a black vertical line (`divide-x`), with space between them for clarity.

Portal Page Component

The `SignPortal` component acts as a dynamic authentication gateway, allowing users to toggle between the **Sign In** and **Sign Up** forms with smooth animated transitions.

8.5.1 Tools & Libraries Used:

1. **React Router (`react-router-dom`)**
 - Used for navigation and routing between pages (e.g., clicking the logo navigates to the homepage).
2. **Framer Motion (`motion/react`)**
 - Handles animations and transitions between the login and signup panels.
 - `AnimatePresence` enables exit animations.
 - `motion.div` is used for animating layout changes and opacity.
3. **Tailwind CSS**
 - Provides all the styling for layout, responsiveness, and colors.
 - The portal is centered using flex utilities and styled with shadows, rounded corners, and padding.
4. **Custom Context Hook (`useSignPortalContext`)**
 - A custom React hook that manages whether the user is currently on the "sign-in" or "sign-up" state.
5. **Component Composition**
 - **SignIn** and **SignUp** components are rendered conditionally based on the current state.
 - A shared **Logo** component is shown above the form.

8.6 1.The signin

component is a form-based component used for user authentication in the "Sign In" portal of the FastAF application. It is designed to handle user input for logging into the platform using a username (email or phone number) and password.

8.6.1 Key Features:

- **Form Validation:** Utilizes react-hook-form with zodResolver to handle form validation based on the signInSchema.
- **Form Fields:** Includes fields for username (email or phone number) and password.
- **Form Submission:** On form submission, the data is processed using the useSignInV2 hook, which manages the sign-in process.
- **Pending State:** A Spinner component is displayed when the sign-in request is being processed.

8.6.2 Component Breakdown:

- **Form Controls:**
Each input field (username and password) is wrapped in FormField, FormControl, FormItem, and FormLabel to manage validation and display errors.
- **Button:** The submit button (Sign In) is disabled while the request is pending (isLoading), and a loading spinner (Spinner) is displayed during this time.
- **Link to Sign Up:** If the user does not have an account, they can navigate to the Sign Up page by clicking the Sign up now link.

Certainly! Let me provide a more detailed breakdown of the `SignIn` component, explaining its structure, dependencies, hooks, form logic, and overall functionality.

8.6.2.1 1.3 Imports

The `SignIn` component relies on several libraries and custom components:

- **React Hook Form:**
 - `useForm`: Hook from react-hook-form to manage form state.
 - `zodResolver`: Resolver that integrates the Zod validation library with React Hook Form.
 - **Form components:** `Form`, `FormControl`, `FormItem`, `FormField`, etc. These are custom components built to handle form elements in a consistent way.
- **Zod:**

- **signInSchema**: A Zod schema for validating form inputs (i.e., username and password).
- **Custom Components**:
 - **Button**: A custom button component used to submit the form.
 - **Spinner**: A custom spinner component shown when the form is submitting.
 - **Input**: A custom input component.
 - **SignPortal Context**: `useSignPortalContext` is a hook that helps manage the state of the sign-in and sign-up portal.
- **Services**:
 - **useSignInV2**: A custom hook for handling the sign-in mutation, which likely performs the API call to authenticate the user.

8.6.2.2 1.4 Form Setup

- The `useForm` hook is initialized with default values for username and password, and the `zodResolver` is used to handle validation via the `signInSchema`.

Form Fields:

- **Username**: The username field accepts either an email or phone number. It uses the `Input` component and displays validation error messages via `FormMessage`.
- **Password**: The password field is of type `password` for secure entry. It also has validation and error handling.

8.6.2.3 1.5 Validation Schema (`signInSchema`)

The `signInSchema` is a Zod validation schema that is applied to the form fields. Here's an example of what the schema might look like (you would define it separately):

- **username**: Must be a non-empty string and must match an email format.
- **password**: Must be a non-empty string with a minimum length of 6 characters.

8.6.2.4 1.6 Form Submission

When the form is submitted, the `onSubmit` function is triggered:

- **signIn**: The `signIn` function is called from the `useSignInV2` hook. This is likely an API request to authenticate the user with the backend. It takes the form data (username and password) as its input.

8.6.2.5 1.7 Pending State

- The `Button` is disabled during the form submission (when the `isPending` flag is `true`), and a `Spinner` component is displayed to indicate loading
- **isPending:** This state tracks if the form submission is in progress. If `isPending` is `true`, the button is disabled, and the spinner is shown. Once the sign-in process completes, the button is re-enabled.

8.6.2.6 1.8 Redirect to Sign-Up

- If the user does not have an account, they can navigate to the "Sign Up" page by clicking on the "Sign up now" link.
- **setPortalParam:** This function is from the `useSignPortalContext` hook. It helps manage the state of which portal (sign-in or sign-up) should be shown.

8.6.2.7 1.9 Error Handling

Each form field (`username` and `password`) displays error messages via the `FormMessage` component if validation fails.

This component will display validation errors that come from the Zod schema, helping the user know what needs to be corrected.

8.6.2.8 1.10 Responsive Design

The `SignIn` component is responsive and can adapt to different screen sizes. The form uses Tailwind CSS classes to adjust spacing and alignment, such as `space-y-6` for vertical spacing between form elements.

8.7 2- The sign up

component is a user registration form built with React, using `react-hook-form` for form management, Zod for schema-based validation, and custom UI components for styling. It also interacts with a custom authentication service for handling the sign-up process.

8.7.1 Breakdown of the sign up Component:

8.7.1.1 2.1 Imports

- **React Hook Form:** Handles form state management and validation.
 - `useForm`: Initializes the form with default values and sets up validation.
 - `FormField`, `FormItem`, etc.: Custom UI components for wrapping each form element.
 - `zodResolver`: Integrates Zod for schema validation.
- **Zod:** The `signUpSchema` is a Zod schema used for validating the form data, ensuring fields meet certain criteria.
- **Custom Components:**
 - `Button`: A custom button component for form submission.
 - `Spinner`: A custom spinner component displayed when the form is submitting.
 - `Input`: A custom input field component.
- **Hooks & Services:**
 - `useSignPortalContext`: A context hook used for managing the sign-in and sign-up flow.
 - `useSignUp`: A custom hook for handling the sign-up process.

8.7.1.2 2.2 Form Setup

The `useForm` hook is used to set up form state, validation, and submission logic:

- **mode:** "onChange" means validation will run as the user types.
- **resolver:** Uses Zod to validate the form data against the `signUpSchema`.
- **defaultValues:** Default values for form fields (email, password, repassword, username).

8.7.1.3 2.3 Form Fields

Each form field is wrapped with `FormField`, and validation error messages are shown using `FormMessage`.

- **Username:** A simple text input field for the user's name.
- **Email:** An email input field for the user's email address.
- **Password:** A password input field for the user's password.
- **Confirm Password:** A field for confirming the password, to ensure the user entered it correctly.

8.7.1.4 2.4 Form Submission

The form submission is handled by the `onSubmit` function:

- **repassword:** The `repassword` field is excluded from the data sent to the `signUp` function since it's just used for validation.
- **signUp:** This function, coming from `useSignUp`, handles the actual sign-up process, which likely includes sending data to the server for account creation.

8.7.1.5 2.5 Loading State

A loading spinner is displayed while the sign-up process is ongoing, using the `isSigningIn` state

isSigningIn: This boolean value is set by the `useSignUp` hook to track whether the sign-up process is in progress. If it's `true`, the button is disabled, and the `Spinner` component is shown.

8.7.1.6 2.6 Navigation to Sign-In

If the user already has an account, they can navigate to the sign-in page by clicking the "Login" link.

setPortalParam: This function is provided by `useSignPortalContext` to manage the current state of the portal (whether the sign-in or sign-up form should be shown).

8.7.1.7 2.7 Responsive Layout

The form is wrapped in a `div` with a `w-96` class to set its width. You can adjust this depending on the layout needs for your application. The rest of the layout is managed using Tailwind CSS utility classes, ensuring the form looks good on different screen sizes.

Pick pharmacy Page Component

It looks like you've created a dialog component for creating a pharmacy, which includes a form with various fields like pharmacy name, address, whether it's a branch, and expiry threshold. The form is using `react-hook-form` with validation from `Zod`, and it integrates with your pharmacy API service to create the pharmacy.

8.7.2 Key Features:

- **Dialog Component:** The pharmacy creation form is wrapped in a dialog for a clean, modal-style interface.
- **Conditional Fields:** The form handles whether the pharmacy is a branch or not. If it's a branch, the main branch ID is displayed and set as disabled.
- **Form Validation:** `zod` is used for form validation via `react-hook-form`'s `zodResolver`.
- **API Integration:** The `createPharmacy` function from your mutations service is called when the form is submitted.
- **Onboarding Background:** The background image (`onboardingSrc`) gives the page an attractive onboarding screen appearance.
- **Fetching Pharmacies:** It uses the `useGetUserPharmacies` hook to fetch the pharmacies associated with the user. If the request is still pending, a `Spinner` component is shown.
- **Displaying Pharmacies:** If there are pharmacies available, they are displayed using the `PharmacyItem` component. The data is mapped and rendered as individual pharmacy items.
- **Create Pharmacy:** If no pharmacies are found, the user is given an option to create a new pharmacy using the `CreatePharmacyDialog` component, which lets users create a pharmacy directly from this screen.
-

8.7.3 Possible Improvements:

- **Loading State:** You have a loading state handled by `isLoadingPharmacy` which shows a spinner when the form is being submitted. Ensure that the spinner component is visible and functional.
- **Form Field Validations:** If more detailed validation is needed (such as for the address field), you can extend the `Zod` schema.
- **UI Feedback:** Consider adding a success/error message after submitting the form, or redirecting the user back to the pharmacy list after a successful creation.

Dashboard Page Component

For your dashboard, you want to display: Inventory of medicines Sales information Employee information

8.8 1-CreateEmployeeForm

`CreateEmployeeForm` is a component that handles the creation of a new employee. It uses `React Hook Form` for form handling, integrates `Zod` for validation with the `createEmployeeSchema`, and displays a set of fields for input including `username`, `email`, `password`, `age`, `gender`, `salary`, and `shiftId`.

Features:

- **Username, Email, and Password Fields:** Users can input details for the new employee, with the `email` field being of type `email` and the `password` field being `password`.
- **Age and Salary:** Numeric fields for age and salary.
- **Gender Selection:** Uses a `RadioGroup` component to select either `male` or `female`.
- **Shift Selection:** Allows selecting a shift from a list of available pharmacy shifts. The `Select` component is used for dropdown functionality, with options fetched dynamically.
- **Form Submission:** Once the form is validated, the employee is created using the `createEmployee` mutation. The button is disabled while an employee is being created.

8.8.1 EditEmployeeForm

`EditEmployeeForm` allows you to edit the details of an existing employee. Similar to `CreateEmployeeForm`, it uses `React Hook Form` and `Zod` validation (`editEmployeeSchema`). The default form values are populated with the employee's existing data.

Features:

- **Username, Email, and Salary Fields:** Similar to the create form, these fields allow for updating the employee's details.
- **Age and Gender:** The age can be updated, and the gender is selectable using a `RadioGroup`.
- **Shift Selection:** You can modify the shift assigned to the employee by selecting from a list of available shifts.
- **Form Submission:** On submitting the form, the `editEmployee` mutation is called, and the changes are saved.

8.8.2 DeleteEmployee

DeleteEmployee is a confirmation modal for deleting an employee. It provides two buttons: Cancel and Delete. When confirmed, it triggers the deletion of the employee and closes the modal.

Features:

- **Cancel Button:** Closes the dialog without taking action.
- **Delete Button:** Calls the `onDelete` function to remove the employee from the system and closes the modal after confirmation.

8.8.3 ShiftsCard

ShiftsCard displays a list of shifts in a card format. It uses the `usePharmacyShifts` hook to fetch shift data and displays the shifts in a scrollable area. Each shift is rendered using the `ShiftItemWithContext` component, which allows for the deletion of a shift.

Features:

- **Shift List:** Displays all shifts retrieved from the API.
- **Loading State:** A skeleton loader is shown while the shifts data is being loaded.
- **Delete Shift:** Each shift can be deleted by clicking the delete button, which triggers the `deleteShift` mutation.

8.8.4 DataTableRowActions

DataTableRowActions renders the action buttons for each row in the data table. It includes options for editing and deleting an employee.

Features:

- **Edit Action:** Opens the `EditEmployeeForm` dialog when triggered.
- **Delete Action:** Opens the `DeleteEmployee` dialog when triggered.
- **Dropdown Menu:** A dropdown menu containing the actions (`Edit` and `Delete`) for each row.

8.8.5 Shifts and Employees Data Fetching

The application uses hooks like `usePharmacyShifts` and `usePharmacyEmployees` to fetch data about shifts and employees from the pharmacy API.

- `usePharmacyShifts`: Fetches a list of shifts assigned to the pharmacy, including shift names, IDs, and start/end times.
- `usePharmacyEmployees`: Fetches a list of employees along with their details (e.g., name, shift, salary).

These hooks are used in various components like `ShiftsCard` and `CreateEmployeeForm` to dynamically populate UI elements like dropdowns and tables.

8.8.6 Pharmacy Shifts and Employee Management

This section explains the interrelationship between employees and shifts. Each employee can be assigned to a shift, and managing shifts is a core part of the system. The UI allows administrators to create shifts, assign employees to shifts, and make modifications as needed.

Conclusion

The system provides a comprehensive interface for managing employees and shifts within a pharmacy. From creating new employees, assigning shifts, editing details, and deleting employees, the application provides a complete suite of features for managing staff efficiently.

8.8.7 ExpiryPage Component

This component manages the logic for the expiry page of a pharmacy, displaying a data table of drugs and providing filtering options.

8.8.7.1 1. `useParams` and `useSearchParams`

- `useParams`: This is used to extract the `pharmacyId` from the URL (e.g., `/pharmacy/:pharmacyId/expiry`). The `pharmacyId` is used to fetch data specific to a particular pharmacy.
- `useSearchParams`: This is used to manage URL query parameters, allowing dynamic updates to the search query, filter type, pagination (page, size), and so on.

8.8.7.2 2. Query Parameters

- **search:** The search term entered by the user (if any), which is used for filtering the drugs by name or other attributes.
- **filter:** Determines which drugs to show based on their expiry status. The possible values are:
 - "EXPIRED" – Drugs that are expired.
 - "NOT_EXPIRED" – Drugs that are not expired.
 - "APPROACHING_EXPIRY" – Drugs that are nearing their expiry date.
- **page and size:** Pagination settings to control the data size and which page of the results to fetch.

8.8.7.3 3. Data Fetching with React Query

- **useQuery:** The `useQuery` hook is used to fetch data from the pharmacy API (`getSearchPharmacy` function). The query is dependent on `pharmacyId`, `filter`, `page`, `size`, and `search` query parameters.
- **isLoadingDrugs:** A flag indicating whether the data is still being fetched.

8.8.7.4 4. Rendering the Page

- A title "Expiry Page" is rendered.
- **SearchInput:** A component for searching drugs by name or other relevant attributes.
- **SelectFilter:** A dropdown for selecting the expiry status filter (Not expired, Expired, Near Expiry).
- **DataTable:** Displays the drugs data in a tabular format, with loading and error handling capabilities.

8.8.8 Columns Configuration for the Data Table

The `columns` array defines the columns for the `DataTable` component, specifying what data will be displayed in each column and how they should be sorted.

8.8.8.1 Columns Breakdown:

1. **Name Column:**
 - Displays the name of the drug (`drug.name`).
2. **Form Column:**
 - Displays the form of the drug (e.g., tablet, liquid) from `drug.form`.
3. **Quantity Column:**
 - Displays the stock of the drug from `stock`.

- It uses a custom header with sorting functionality via `sortedHeader`.
4. **Expiry Date Column:**
- Displays the expiry date of the drug from `expiryDate`.
 - This column also uses `sortedHeader` for sorting the drugs based on their expiry date.

8.8.8.2 *sortedHeader Utility*

- `sortedHeader` is used to dynamically generate the header for sortable columns, enabling users to sort drugs by quantity or expiry date.

8.8.9 header

8.8.10 CommandMenu Component

This component provides a search bar where users can quickly type commands or search for features in the application.

8.8.10.1 *Key Features:*

- **Keyboard Shortcut:** Pressing `⌘J` (or `Ctrl+J` on Windows/Linux) opens the `CommandMenu`. This is handled via a `useEffect` hook, which listens for the `keydown` event and toggles the menu open/close state.
- **UI:**
 - Displays a search icon (Search) and a prompt (Type a command or search...).
 - When the user types, the `CommandInput` component allows the input of commands or search terms.
 - **Command List:** Includes different **CommandItems** such as `Calendar`, `Search Emoji`, `Calculator`, and settings-related options such as `Profile`, `Billing`, and `Settings`.
 - **Command Shortcuts:** Shows the associated keyboard shortcut for each command (e.g., `⌘P` for Profile, `⌘B` for Billing, `⌘S` for Settings).

8.8.10.2 *Usage of Components:*

- `CommandDialog`: This is the dialog that shows when the command menu is open.
- `CommandInput`: An input field for typing commands.

- `CommandList`: The list of command options (with separate groups for Suggestions and Settings).
- `CommandItem`: Individual items in the command list, each with an icon and description.

8.9 inventory

8.9.1 Columns Definition

- The columns are defined using `ColumnDef<InventoryItem>` for the main inventory data and `ColumnDef<InventoryItemTest>` for inventory test data.

8.9.1.1 Key Columns for `InventoryItem`:

- **id**: Displays the drug ID.
- **name**: Displays the drug name with a custom `cell` renderer to wrap it in a `div` with a max width.
- **form**: Displays the form of the drug (e.g., tablet, syrup).
- **price**: Displays the price with a dollar sign (\$) and custom formatting.
- **expiryDate**: Displays the drug's expiry date formatted as `MMMM, d`.
- **stock**: Displays the available stock.

8.9.1.2 Key Columns for `InventoryItemTest`:

- **ID**: Drug ID.
- **Name**: Drug name.
- **Type**: Drug form.
- **Quantity**: Quantity available.
- **Needed**: The quantity needed.
- **Shortage**: The quantity shortage (custom column with a filter function).
- **Actions**: Dropdown menu with options to **copy drug ID**, **edit**, or **delete** the inventory item.

8.9.2 Filtering Logic:

The **inventoryStatusFilter** function filters the inventory based on different status options:

- **Available**: Quantity > 0 and no shortage.
- **Shortage**: The drug has a shortage.
- **Unavailable (Shortage)**: The drug is unavailable with a shortage.
- **Unavailable**: The drug has no stock.

This filter is applied to the **status** column in the **testColumns** array and allows users to quickly filter the inventory based on stock and shortage status.

8.9.3 Inventory Page with Dynamic Data Fetching:

The `Inventory` component fetches the inventory data dynamically from the backend API using **React Query**. It uses the following features:

- **Search Input:** A search input to search for drugs by name.
- **Filter:** A dropdown filter to filter inventory based on its status (Available, Shortage, Unavailable, etc.).
- **Pagination:** The table supports pagination, with the `page` and `size` being fetched from the URL search parameters (`useSearchParams`).
- **Data Fetching:** The `useQuery` hook fetches data based on the `pharmacyId`, `search`, `filter`, `page`, and `size` parameters.

8.105-pharmacy

8.10.1 DeletePharmacy

- This component provides functionality to delete a pharmacy, showing a confirmation button while it's being deleted.
- It uses a button to cancel the action or delete the pharmacy.

8.10.2 DeleteShift

- Similar to the `DeletePharmacy` component, this one is for deleting a shift.
- It displays a loading state while the shift is being deleted and shows a confirmation dialog.

8.10.3 EditPharmacyForm

- This form allows editing of pharmacy details, like the name, address, and expiry threshold.
- It uses React Hook Form with validation from Zod, and upon submission, the data is updated using the `editPharmacy` mutation.

8.10.4 PharmacyItem

- Represents an individual pharmacy with its name, address, and creation date.
- It includes options for deleting or editing the pharmacy, and allows navigation to a detailed view of the pharmacy.

8.10.5 PharmacyItemWithMenu

- A more advanced version of `PharmacyItem` that includes a dropdown menu for actions like delete and edit.
- It conditionally opens dialogs for confirming the delete or editing actions.

8.10.6 ShiftItem

- Represents an individual shift, showing the name and time range.
- It includes a delete button and handles loading states when deleting the shift.

8.10.7 ShiftItemWithContext

- Similar to `ShiftItem` but with additional context for handling delete actions via a dropdown menu.
- It opens a confirmation dialog for deletion and provides more options through the dropdown menu.
-

8.11 Android implementation

8.11.1 What is Android?

Android is an open-source operating system developed by **Google** for building high-performance mobile applications. It powers a wide range of smartphones, tablets, and other smart devices. Android apps are built using **Java or Kotlin**, and the platform provides a robust framework for designing responsive and feature-rich user interfaces. Unlike some platforms, Android apps run on the **Android Runtime (ART)**, offering optimized performance and efficient memory management. The system is highly customizable and supports deep integration with device hardware and sensors. Android uses **XML** for UI design and integrates with powerful tools like **Android Studio**. With its global reach and flexibility, Android is the most widely used mobile OS in the world.

8.11.2 Why Android?

We chose **Android** as the development platform for our project because of its **high performance, broad device compatibility, and mature ecosystem**. Android powers the majority of smartphones worldwide, making it ideal for achieving **widespread accessibility and user reach**. It runs smoothly on a wide range of devices—from low-end phones to high-performance flagships—thanks to its scalable design and optimized system architecture.

Android apps run on the **Android Runtime (ART)**, which enhances performance using **ahead-of-time (AOT)** and **just-in-time (JIT)** compilation. This ensures **fast app launches, smooth animations, and efficient memory usage**, even on lower-spec devices. The platform also supports **hardware acceleration** and uses efficient rendering engines to deliver a responsive and engaging user experience.

A major advantage of Android is its **long-established global developer community**. Since Android has been around for over a decade, most common development problems already have documented solutions, tutorials, or libraries available online. This **reduces development time**, helps with troubleshooting, and ensures that developers rarely face an issue alone.

In addition, Android offers deep integration with hardware features like **camera, sensors, GPS, and storage**, which is essential for building modern, real-world mobile applications. Its **powerful tools** (such as Android Studio), rich documentation, and open-source nature make it a robust and future-proof choice for mobile app development.

8.11.3 APPLICATION ARCHITECTURE :

MVVM (Model-View-ViewModel) is an architectural pattern that helps separate the **user interface (UI)** from the **business logic and data handling**. This separation improves the **testability, maintainability, and scalability** of the application.

The project follows the MVVM pattern to organize code cleanly, reduce coupling, and ensure each component has a distinct responsibility:

. Model:

- Represents the **data** and **business logic** of the application.
- Responsible for **retrieving**, **storing**, and **processing** data from local databases (e.g., Room) or remote sources (e.g., Retrofit APIs).
- Example: Data classes, repository classes, API response handlers, Room database classes.

. View:

- Responsible for **displaying UI** and **handling user interaction**.
- Implemented as **Activities**, **Fragments**, or **XML layouts** in Android.
- The View should be as **passive as possible**, only reflecting the data it receives from the ViewModel.
- Example: LoginActivity, MainActivity, RecyclerView adapters, etc.

. ViewModel:

- Serves as a **middle layer** between the View and Model.
- Manages **UI-related data**, handles **business logic**, and survives **configuration changes** (like screen rotation).
- Uses **LiveData** or **StateFlow** to expose data to the View in an observable way.
- Calls the Repository to fetch data and prepares it for display.
- Example: LoginViewModel, OrderViewModel, etc.

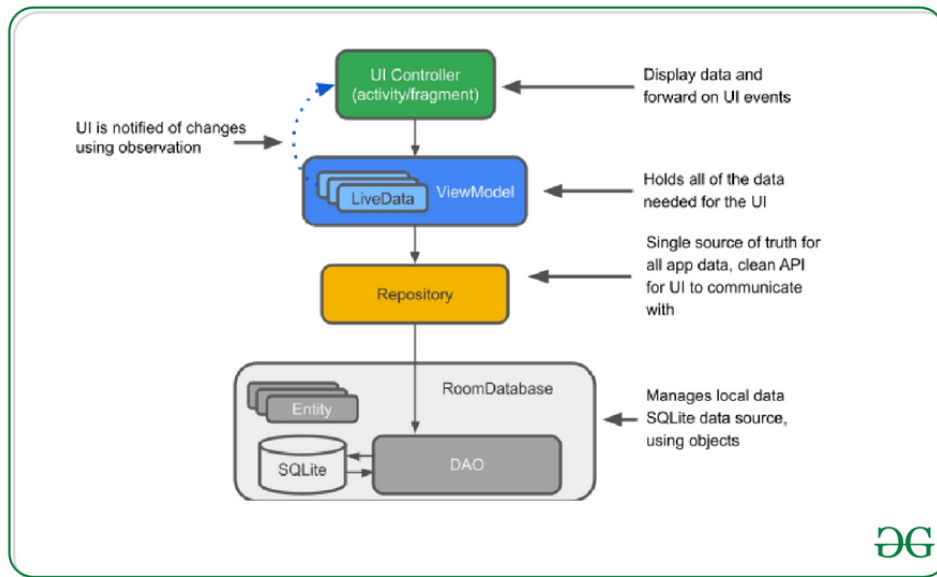


Figure 8-1 view model

- Project Structure :

Table 8-13 project structure

Folder	Description
data	Contains the data layer of the app. Includes:
	- ApiManager: Manages API requests and configuration
	- WebServices: Defines endpoints and handles network calls
	- SocketManager: Manages WebSocket connections ai
core	Contains reusable utilities and helpers shared across the application.
	Examples: constants, formatting utilities, error handlers, etc.
features	Contains feature-specific code such as:

Folder	Description
	- Activity classes (UI Views)
	- ViewModel classes for business and UI logic
	- Any additional data or logic tied to a specific feature/module

8.11.4 Advantages of This Architecture :

1. **Separation of Concerns**
Each layer (UI, business logic, data) has a distinct responsibility, making the codebase easier to maintain and understand.

2. **Feature Modularity**
Each feature is self-contained with its own view, model, and controller. This makes it easier to test, debug, or scale features independently.

3. **Code Reusability**
Shared components like buttons or input fields live in widgets/ and utils/, allowing for reuse across multiple screens or features.

4. **Scalability**
As your app grows, new features can be added without affecting existing ones, since each is in its own module-like folder.

5. **Testability**
By isolating ViewModels and business logic in controllers, you can write unit tests without involving UI.

6. **Clean Architecture Alignment**
This structure loosely follows **Clean Architecture principles**, promoting maintainable and loosely coupled code.

7. **Ease of Collaboration**
Multiple developers can work on separate features or core components simultaneously without conflict.

8. **Improved Code Navigation**
Logical grouping helps developers quickly locate files related to a specific feature or shared utility.

8.11.5 STATE MANAGEMENT :

architectural Pattern: MVVM with MVI Concepts

The application adopts the **MVVM (Model-View-ViewModel)** architectural pattern as the foundation for organizing code structure, ensuring clear separation of concerns, enhanced maintainability, and scalability. Additionally, **MVI (Model-View-Intent)** concepts are selectively integrated to improve state management and unidirectional data flow for complex features

MVVM Layer Breakdown:

1.Model Layer

- Represents the **data layer**, which includes API managers, repositories, and data models.
- Responsible for all data operations such as fetching data from remote APIs, local databases, or WebSocket connections.
- Contains classes such as:
 - ApiManager (handles RESTful API requests)
 - WebSocketManager (handles real-time socket communication)

- Data models used to parse and structure response data.

2. View Layer

- Contains **Activities** and **Fragments** responsible for rendering the user interface.
- Observes state from the ViewModel and updates the UI accordingly.
- Interacts with the user through input fields, buttons, and navigation events.
- Remains as simple as possible, containing no business logic.

3 ViewModel Layer

- Acts as the **bridge** between the View and Model layers.
- Holds all business logic and manages UI-related data using LiveData or StateFlow.
- Emits updated UI state to the View reactively.
- Translates user events into business logic and then to view state updates.

Integration of MVI Concepts:

While MVVM remains the primary architecture, several MVI principles are integrated to enhance state management in complex scenarios:

MVI Concept	Applied in the Project
Intent (User Actions)	User actions like button clicks are modeled as events sent to the ViewModel.
Single Source of Truth	The UI state is maintained as a single data object (sealed class or data class), ensuring clarity and consistency.
Immutable State	UI states are updated using Kotlin's <code>copy()</code> function to create new immutable instances.
Unidirectional Flow	Data View → Intent → ViewModel → State → View, ensuring a predictable update cycle.

8.11.6 How It Works?

1. **User** **Interaction:**
The user performs an action (e.g., clicks a button or types in a text field).
2. **Event** **to** **ViewModel:**
The view sends this interaction as an **event or intent** to the ViewModel.
3. **Business** **Logic** **in** **ViewModel:**
The ViewModel processes the event, interacts with the Model if needed (e.g., calls API), and updates the internal UI state.
4. **State** **Emission:**
A new **UI state** is emitted via StateFlow or LiveData, and observed by the View.
5. **UI** **Update:**
The View updates itself based on the new state, reflecting changes like loading indicators, error messages, or successful results.

- **Advantages of MVVM + MVI Architecture :**

1. **Separation of Concerns**

- Divides the app into distinct layers (Model, View, ViewModel), making the codebase easier to understand and manage.

2. **Improved Maintainability**

- Changes in UI or business logic can be made independently, reducing the risk of introducing bugs.

3. **Testability**

- ViewModels are UI-independent and easy to test using unit tests, improving code reliability.

4. Reactive and Predictable UI Updates

- Using LiveData or StateFlow, the UI automatically reflects the latest state without manual intervention.

5. Unidirectional Data Flow (MVI Concept)

- Makes state changes easier to track and debug, especially in complex scenarios.

6. Single Source of Truth

- All UI state is centralized in one place, ensuring consistency and avoiding unexpected behaviors.

7. Scalability

- Easily accommodates new features or screens without disrupting existing functionality.

8. Reusability of Components

- Shared ViewModels, states, and models can be reused across different parts of the app, reducing code duplication.

8.11.7 USER INTERFACE DESIGN USING XML:

1 XML-Based Layout Design

- All screens and components are designed using XML files under the res/layout directory.
- Layouts use components such as ConstraintLayout, LinearLayout, RecyclerView, TextView, ImageView, and custom views for structured, responsive design.
- UI components are styled using themes and custom styles defined in styles.xml.

2 Responsive Design

- The app uses **ConstraintLayout** for flexible UI that adapts to different screen sizes and resolutions.
- Dimensions and spacing are handled using dp, sp, and resources defined in `dimens.xml` to maintain consistency across devices.

3 User Experience (UX) Considerations

- **Interactive Elements** like buttons and input fields follow material design principles.
- **Feedback Mechanisms** such as loading indicators, error messages, and success confirmations are implemented to enhance usability.
- **Navigation** is intuitive, using `NavController` or intent-based flows between activities/fragments.

- **SCREENS:**

Each screen in the application serves a specific purpose and is designed to ensure a seamless user experience. The screens are structured to adhere to best practices in UI/UX, including responsive layouts, clear navigation, and consistent design patterns.

- **Login Screen:**

The login feature is designed using the **MVVM architectural pattern**, providing a clean separation of concerns. It handles user authentication by communicating with a secure API and managing **access** and **refresh tokens**, with proper **error handling** and **security considerations** to ensure safe user authentication and data integrity.

- **Architecture Overview:**

1. **View (LoginActivity/LoginFragment):**
Displays the UI, observes login state, and shows loading/errors/success messages.

2. **ViewModel (LoginViewModel):**
Holds login state, handles logic for calling the API, validating inputs, and exposing LiveData/StateFlow for the View to observe.

3. **Model (Repository & Data Layer):**
Interacts with the API service using Retrofit and securely stores the tokens using EncryptedSharedPreferences.

- Login Flow :

1. User enters email and password in the UI.

2. ViewModel validates inputs.

3. If valid, the ViewModel calls the Repository.

4. The Repository sends a login request to the API.

5. On success:

- Stores access_token and refresh_token securely.
- Navigates to the main screen.

6. On failure:

- Displays appropriate error messages (e.g., invalid credentials, server error, no internet).

- Token Handling:

1. **Access Token:**
Used to authenticate API requests. Stored securely and attached to every request via OkHttp Interceptor.

2. **Refresh** **Token:**
When the access token expires, the refresh token is used to request a new access token without re-logging in.

3. **Security** **Storage:**
Tokens are stored using **EncryptedSharedPreferences** to prevent plain-text exposure.

- Error Handling:

1. **Validation** **Errors:**
Handled in ViewModel before calling API (e.g., empty fields, invalid email).

2. API Errors:

- 401 Unauthorized → Trigger token refresh or logout.
- 400 Bad Request → Show form errors.
- 500 Server Error → Display generic error message.

3. Network Errors:

- Handled with try-catch blocks and connectivity checks.
- Appropriate message shown if offline.
- Home Screen:

The **Main Screen** serves as the central hub of the application. Once a user successfully logs in, they are navigated to the main screen, which provides access to all major features such as search, cart, profile, order management, and more.

- Navigation Role:

1. Acts as a **controller or gateway** to all feature-specific screens.

2. May use **Bottom Navigation**, **Navigation Drawer**, or **Tabs** to switch between major sections.

3. Handles **fragment transactions** or **activity transitions** depending on the feature.

- Responsibilities:

1. Display accessible UI for navigating to:

- Search screen
- expiry
- Cart
- Order
- Settings or Profile
- Architecture and Reusability:

1. Shared components like Toolbar, BottomNavigationView, or AppBar are reused across feature fragments.

2. Logic is separated in MainViewModel, ensuring main activity remains clean and focuses only on UI-related logic.

- Cart Screen :

The Cart module allows users to collect and manage selected drugs before submitting or processing them. It supports two input methods: AI-based detection using Camera and manual search-based selection.

- Interaction Methods :
- *AI Detection via Camera:*

1. When using the **Camera screen** connected to an AI model through **Socket communication**, drugs are identified in real-time.

2. **Single Touch:**

- Adds the detected drug to the cart.
- Triggered by a light tap on the screen after a successful detection.

3. **Long Press:**

- Opens the **Drug Details** screen with more information.

- Allows users to review the drug before adding it to the cart.
- *Manual Search:*

1. User inputs drug name/type into a **Search field**.

2. Search results include drug name and type.

3. **Click on Drug Name:**

- Instantly adds the selected drug to the cart.

4. **Open Drug Details:**

- Tapping on another part of the item (e.g., a button or info icon) opens the drug details for inspection.
- Order Screen :

The **Order Management** module supports pharmacy staff in tracking shortages and generating purchase orders efficiently. Built using **MVVM + MVI architecture**, it ensures clean separation of concerns and responsive UI behavior.

- Architecture:
- **MVVM Role:** OrderViewModel and PreViewViewModel handle business logic, session/token checks, and LiveData/StateFlow for reactive updates.
- **MVI Role:** Handles user intents (e.g., tap, submit, change amount), and updates view state accordingly (e.g., CartUpdated, OrderConfirmed).

- Key Screens:

1. **OrderActivity:**

- Displays shortage drugs.
- User inputs order quantities.

2. **PreViewOrderActivity:**

- Shows a summary of ordered drugs.
- Allows naming and confirming the order.

3. **New Page: OrderReviewActivity:**

- Displays **incoming orders** from other users.
- Allows **accepting** or **declining** each order.
- Updates order status on the server accordingly.

- **Features Overview:**

1. **Shortage Display:** Fetches real-time stock shortages.
2. **Live Quantity Entry:** Users enter how much of each drug to order.
3. **Preview Screen:** Review and edit items, add a custom order name.

4. **Confirmation Handling:**

- Only valid orders (at least one drug) are accepted.
- Confirmation persists across the session.

5. **Review Page for Managers:**

- Accept or decline received orders.
- Confirmed orders are processed for inventory update.

- **Expiry Screen :**

he **Expiry Management Module** is a critical component of the pharmacy inventory management system. It enables pharmacists and medical staff to track the expiration status of drugs, take timely action to remove expired or nearly expired medications from circulation, and ensure that all products dispensed are safe for patient use. This chapter describes the design, implementation, and functionality of this module.

- System Architecture:

The **Expiry Management Module** is built using the **Model-View-ViewModel (MVVM)** architecture, which ensures clear separation of responsibilities and supports scalability, testing, and maintainability

- *View Layer:*

1. **ExpiryActivity**: Acts as the primary user interface. It contains:

- Search input
- Filter chip group
- RecyclerView to show the list of drugs
- Loading spinner and back navigation

2. **ExpiryAdapter**: A custom RecyclerView adapter that binds each ResponseSearchItem to a visually formatted CardView.

- *ViewModel Layer:*

ExpiryViewModel:

Responsible for handling business logic:

- Tracks the selected expiry filter
- Processes user search input (with debounce logic)
- Makes API requests to fetch filtered drug lists
- Updates the UI through LiveData

- *UI Elements & Components:*

1. EditText: For real-time search input
2. ChipGroup & Chips: For selecting expiry filters
3. RecyclerView: For displaying results

4. ProgressBar: Shown during data load
5. CardView: Clean presentation of drug information

- Features Overview :

The **Expiry Management Module** is designed to assist pharmacy staff in tracking and managing the shelf life of drugs within the inventory. By clearly identifying items that are **expired**, **approaching expiry**, or **valid**, this feature ensures that no expired medicine is distributed and that stock is always up-to-date.

This module enhances patient safety, streamlines inventory management, and helps staff take timely decisions regarding restocking, discarding, or postponing drug usage. It combines user-friendly interfaces with robust filtering and search functionalities to provide a seamless experience.

- *Key Functional:*

1. **Filter by expiry state:** "Expired", "Approaching Expiry", or "Not Expired".
2. **Real-time drug name search**, with support for filtering and searching simultaneously.
3. **Card-based display** of each drug's name, form, stock, and expiry date.
4. **Single-selection chip group** ensures only one filter is active at a time.
5. **Live updates** using LiveData and debounced search.

- Reset Filter and Echange :

The **Reset Filter and Invoice Search Module** assists pharmacy staff in efficiently locating and reviewing drug reset records and historical invoices. The module offers a **user-friendly interface** with advanced filtering capabilities, ensuring a smooth and compliant user experience by enforcing business rules

- Key Features :

1. Smart Filtering

- **Date Filter:** Allows filtering by Today, Yesterday, or a Custom Date.
- **Time Filter:** Provides options like Shift Time, All Day, or a Custom Time range.
- **Employee Filter:** Filters by Active Employees or optionally by All Employees.
- **Drug Search:** Users can search by **partial/full drug names**.

2. Search Functionality

- **Drug Name Search:** Supports both **partial** and **full** drug name search.
- **Combined Filtering:** Search can be combined with other filters (Date, Time, Employee) to narrow down results.

3. Invoice Size Limit

- Users are **restricted from retrieving invoices** with more than 16 items.
- If an attempt to fetch such an invoice is made, an error message is shown:

"Cannot retrieve an invoice with more than 16 items."

4. Error Handling

- **No Results:** If no results are found, an **empty state message** is displayed to inform the user.
- **Network Errors:** The system provides a **retry option** to handle connection issues.
- **Invalid Input:** If the user selects invalid filters (e.g., an incorrect date range or time), a **prompt** is shown to reselect valid filters.
- *UI Layout*

1. Filter Panel

- A **filter panel** with easy-to-use components like:
 - **Date Chips:** Quick selection for Today, Yesterday, and Custom Date.

- **Time Dropdown:** Allows the user to select time filters (Shift Time, All Day, Custom).
- **Employee Selector:** A dropdown list of Active Employees or optionally, all employees.

2. Search Bar

- **Text Input** with live search updates as the user types.
- **Debounce Support** to optimize performance and avoid unnecessary API calls while typing.

3. Results View

- The results are displayed using a RecyclerView with the following details:
 - **Drug Name**
 - **Reset Quantity**
 - **Time of Reset**
 - **Employee Name**
 - **Invoice ID**

8.11.8 Architecture :

The architecture of the **Reset Filter and Invoice Search Module** follows a **clean architecture** approach that separates concerns into distinct layers. This modular approach enhances maintainability, scalability, and testability, which is vital for long-term project success. The system is divided into four main layers: **UI Layer**, **ViewModel Layer**, **Domain Layer**, and **Data Layer**.

8.11.9 UI Layer :

The **UI Layer** is responsible for displaying the interface and handling user interactions. This layer comprises the **Activity** or **Fragment**, which serves as the entry point for the user to

interact with the app. The user is able to apply various filters (e.g., date, time, employee, drug) via the search bar and filter components, which collect the input and pass it to the **ViewModel** for processing.

Additionally, the **RecyclerView** displays the filtered data, providing a responsive and interactive interface to the user. The UI is updated in real-time based on the data passed through **LiveData** observed from the **ViewModel**.

Key components of the UI layer:

- **Search Bar & Filters:** Collects user input and sends it to the **ViewModel**.
- **RecyclerView:** Displays the filtered results interactively.
- **LiveData Observers:** Ensures that the UI reacts to changes in data automatically.
- *Domain Layer :*

The **Domain Layer** is where the business logic resides. It encapsulates the operations necessary to retrieve data based on the user-defined filters. This layer includes **Use Cases** and **Repositories**. The **Use Cases** define the specific business operations, while the **Repositories** handle the data fetching from different sources.

Key components of the Domain Layer:

- **Use Cases:** Encapsulate specific business logic, such as filtering and fetching data based on multiple parameters.
- **Repositories:** Abstract the data sources and provide a consistent interface for the **ViewModel** to interact with. The repository handles the logic for accessing both local and remote data sources (e.g., APIs or databases).

By placing the core business logic in the Domain Layer, we ensure that the data and business rules are independent of the UI layer, making the system more robust and adaptable.

8.11.10 Data Flow :

The data flow within the **Reset Filter and Invoice Search Module** follows a clean and reactive model:

1. **User Interaction:** The user interacts with the search bar and filter components in the UI to specify the desired filter criteria (e.g., date range, employee, etc.).
2. **ViewModel:** The **ViewModel** receives the filter parameters from the UI and requests the filtered data from the **Repository** layer.
3. **Repository:** The **Repository** fetches the data from the appropriate source (either from the network or from a local cache) based on the filters applied.
4. **Data Transformation:** The data retrieved from the **Repository** is transformed into a format suitable for the UI.
5. **UI Update:** The **ViewModel** exposes the filtered data via **LiveData**, which the UI observes and updates the displayed results accordingly.

This flow ensures that the user can interact with the system seamlessly, while the underlying architecture handles the data processing and UI updates efficiently.

8.11.11 Functionality :

The module's main functionality revolves around the filtering and display of reset entries based on user-selected parameters. The user can apply multiple filters simultaneously, such as:

- **Date Filter:** Allows the user to filter data based on specific dates or date ranges.
- **Time Filter:** Enables filtering by specific time slots, such as shift times or all-day records.
- **Employee Filter:** Filters records by employee status, such as active or inactive employees.
- **Drug Filter:** Filters records based on the drug name.

Each filter's interaction with the system is handled by the **ViewModel**, which ensures that all filters are applied correctly and that the data is fetched based on the user's selections.

9 Chapter Nine: Drug Detection Model Development and Deployment

9.1 Introduction

This chapter provides a comprehensive account of the entire pipeline for developing and deploying the AI-powered drug detection system that replaces traditional barcode scanning in pharmacies. It covers data collection and cleaning, model training and selection, server implementation, robust error handling, and deployment strategies. Each section explains the rationale behind technology choices, highlights challenges encountered, and details solutions implemented.

9.2 Dataset Collection and Preprocessing

9.2.1 Scraping the Egyptian Drug Database

- Process: Used Python's requests and BeautifulSoup to extract drug names and forms from the official "Egypt Drug Database". Data was loaded into a pandas DataFrame.
- Cleaning: A custom Python script removed duplicates, normalized drug form labels (e.g., "tablet" vs. "Tab"), and enforced consistent casing.
- Reasoning:
 - Official source ensured completeness and accuracy.
 - Python libraries provided rapid development and easy maintenance.

9.2.2 Initial Image Gathering via SERPAPI

- Approach: Queried Google Images through SERPAPI for each drug name.
- Outcome: Collected ~900 images across 46 classes, but faced:
 - Irrelevant or mislabeled images.

- Homogeneous packaging views leading to overfitting risks.
- Decision: Augment with manual collection to improve diversity and real-world relevance.

9.2.3 Manual Photographic Data Collection

- Method: Captured ~2,000 photographs of drug packaging in various pharmacies under different lighting, angles, and occlusion conditions.
- Rationale:
 - Ensured dataset reflects actual use cases.
 - Increased variety for robust object detection.

9.2.4 Annotation and Augmentation with Roboflow

- Annotation: Employed Roboflow's web interface for bounding box labeling and class assignment across both scraped and manual images.
- Version Control: Leveraged Roboflow dataset versions to track annotation changes and augmentations.
- Augmentation Strategies:
 - Geometric transforms: rotation ($\pm 15^\circ$), flips (horizontal/vertical).
 - Photometric adjustments: brightness, contrast, color jitter.
 - Noise addition: Gaussian noise to simulate camera artifacts.
- Benefits:
 - Dramatically expanded effective dataset size.
 - Improved model generalization to unseen conditions.

9.3 Model Training and Selection

9.3.1 Training Configurations

- Framework: PyTorch-based YOLOv5 architecture.
- Hyperparameters:
 - Batch size: 16
 - Learning rate: 0.0003
 - Epochs: 2000
- Variations Tested:
 - Input resolution: 416×416 vs. 640×640
 - Confidence thresholds: 0.25, 0.5, 0.75
 - Different model scales: YOLOv11s, YOLOv11m

9.3.2 Export to ONNX

- Process: Converted the PyTorch model to ONNX format using `torch.onnx.export`.
- Advantages:
 - Platform-agnostic inference.
 - Optimized runtime via ONNX Runtime.
 - Reduced cold-start latency for WebSocket server.

9.4 WebSocket API Server Implementation

To support real-time drug detection, the ONNX model was served via a FastAPI-based WebSocket server. The server handles continuous image streams, inference, and error management.

9.4.1 Technology Stack

- **FastAPI:** Asynchronous, high-performance Python framework for building the WebSocket API.

- **ONNX Runtime:** Provides fast, cross-platform inference with GPU and CPU providers.
- **OpenCV & NumPy:** For image decoding, preprocessing, and array manipulation.
- **Logging:** Python's logging module configured with environment-based log levels.

9.4.2 Server Workflow

- **Connection Establishment:** Client initiates WebSocket handshake to /ws/detect.
- **Frame Reception:** Server awaits binary frames, each representing an encoded image.
- **Preprocessing:** Bytes decoded with OpenCV, converted to RGB, normalized, and batched.
- **Inference:** ONNX model predicts bounding boxes and class probabilities.
- **Post-processing:** Filter by confidence, apply Non-Maximum Suppression (NMS), compute output coordinates.
- **Response:** Pack results into a JSON payload with detection IDs, bounding boxes, and metadata, then send back to client.

9.4.3 Robust Error Handling

To ensure reliability in production, extensive error management was implemented at multiple stages:

- **Connection Errors**
 - **Timeout Handling:** If `websocket.receive_bytes()` times out (default 30s), the server sends a timeout response and continues listening, preventing socket hang-ups.
 - **Disconnects:** Catches `WebSocketDisconnect` to gracefully close resources and log client ID disconnection.
- **Frame Decoding Errors**

- Invalid Data: If image decoding fails (`cv2.imdecode` returns `None`), server logs a warning and returns an error payload without interrupting the session.
- Unsupported Formats: Generic exceptions around image conversion trigger error responses, ensuring the client is aware of format issues.
- Inference Errors
 - ONNX Runtime Crashes: Wrapped `ort_session.run()` in `try/except` to catch model loading or runtime errors, returning error messages and timestamps.
 - Tensor Shape Mismatches: Validated input tensor shapes before inference; if mismatch detected, server returns a descriptive error.
- Post-Processing Errors
 - NMS Failures: Any exceptions during `cv2.dnn.NMSBoxes` are caught; server logs details and continues without bounding box filtering.
 - Bounding Computation: Errors computing box coordinates or class IDs result in skipping that detection and logging the incident.
- General Exceptions
 - A top-level `try/except` around the entire receive-process-send loop catches unforeseen errors, logs stack traces, and sends a generic error message to the client.

9.4.4 Logging and Monitoring

- Structured Logs: JSON-formatted logs include timestamps, client IDs, request IDs, and error details.
- Metrics: Custom counters track frame throughput, average inference latency, and error rates.
- Health Checks: A simple HTTP `/health` endpoint returns server status and model readiness.

9.5 Containerization and Deployment

9.5.1 Dockerization

Dockerfile:

Base image: python:3.10-slim

System dependencies: opencv-python-headless, onnxruntime-gpu

Application code copy and dependency installation via pipenv or requirements.txt.

Entrypoint: uvicorn main:app

9.5.2 Cloud Deployment

- **Platform:** Deployed on a cloud VM with GPU support (e.g., AWS EC2, GCP Compute Engine).
- **Scaling:** Docker Compose or Kubernetes for horizontal scaling under high load.
- **Environment Configuration:** Securely managed via environment variables for model path, thresholds, and logging levels.
- **CI/CD Integration:** Automated build and push of Docker images on GitHub Actions.

9.5.3 Integration with Mobile Application

- **Connection:** Android client establishes WebSocket connection to the deployed server.
- **Frame Streaming:** Camera frames captured via CameraX are encoded and streamed in real time.
- **UI Feedback:** Received detections are overlaid on the preview with colored bounding boxes and interactive touch points.
- **Fallback:** If WebSocket errors persist, client falls back to manual search or barcode input.

9.6 Yolo model results and accuracy

Training and Validation Metrics:

The figure below presents the training and validation performance of the object detection model over 150 epochs. It includes loss curves (box, classification, and distribution focal loss) for both training and validation, as well as key evaluation metrics such as precision, recall, mAP@0.5, and mAP@0.5:0.95. These trends indicate consistent convergence, improved model accuracy, and effective learning over time.

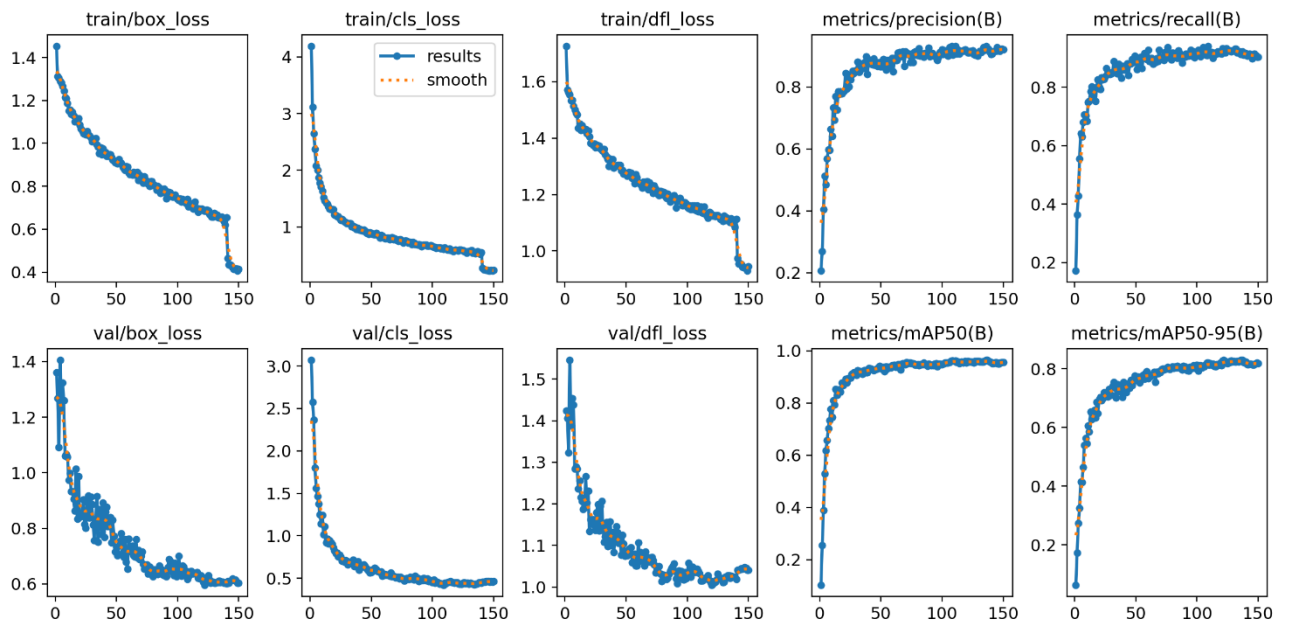


Figure 9-1 yolo model output

9.7 Some validation outputs

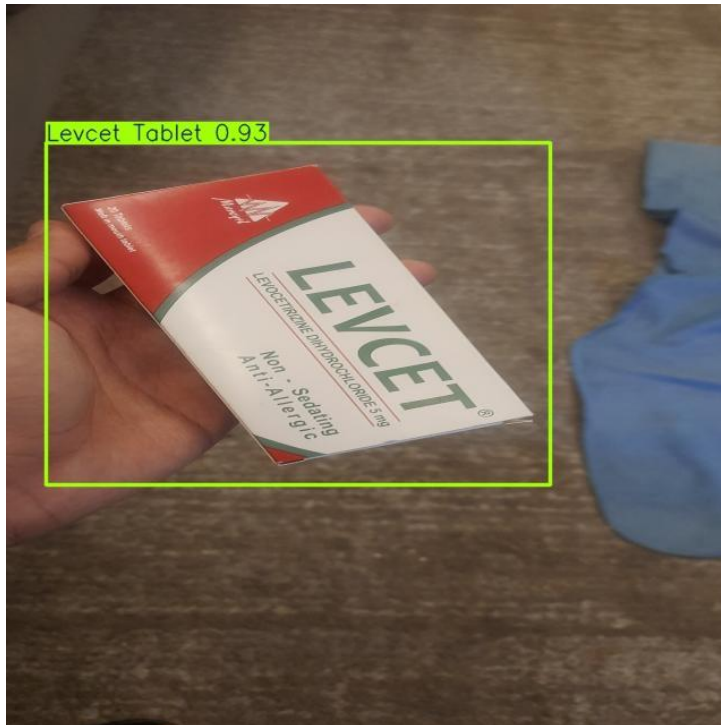


Figure 9-2 model predicted levcet tablet

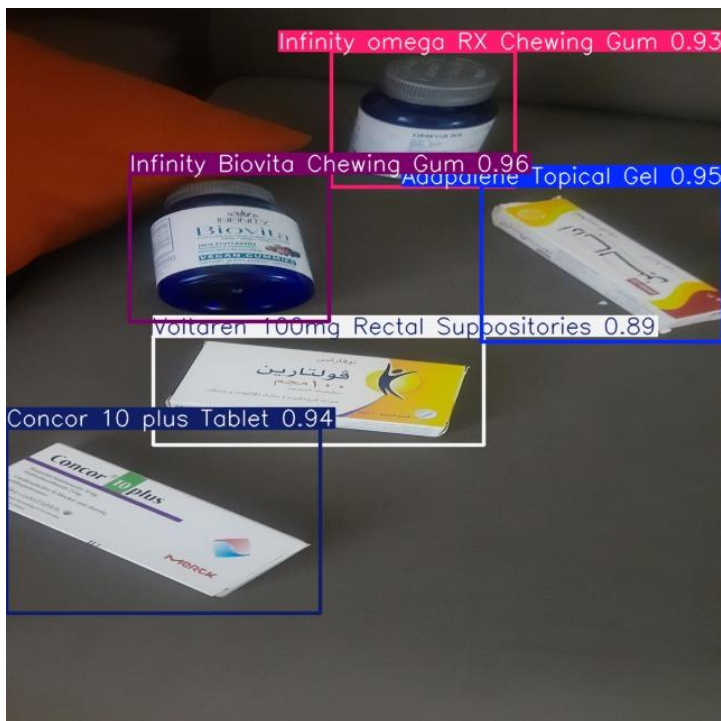


Figure 9-3 model predicted 5 different types of drugs

10 Chapter Ten: Limitations and Challenges

10.1 Introduction

Despite the successful development and implementation of the decentralized pharmacy management system, several limitations and challenges were encountered during the design, development, and deployment phases. These challenges provided learning opportunities and highlighted areas for future improvement.

10.2 Technical Limitations

10.2.1 YOLO Model Detection Accuracy

The real-time object detection feature is powered by a YOLO model trained on a custom dataset of drug packaging. The model performs well under various lighting conditions; however, its accuracy varies with certain camera angles, especially when drug boxes are partially obscured or tilted in certain ways that make it look like another drug. Additionally, the model sometimes struggles to distinguish between visually similar drugs—such as Voltaren 100mg Rectal Suppositories and Voltaren 75mg/3ml Ampoules—due to the limited dataset. This highlights the need for a more comprehensive dataset that includes more samples.

10.2.2 Model Deployment Constraints

The YOLO model is deployed on a cloud server and accessed via a WebSocket API. Although this enables centralized updates and scalability, the system can experience latency under poor internet conditions or heavy load. On mobile devices with weaker hardware, additional lag may occur due to frame encoding/decoding during API communication.

10.2.3 Device Compatibility

The Android app tested on some low-end devices sometimes have performance issues such as camera lag, longer API response times, or even crashes occurred due to limited RAM or CPU power.

10.2.4 Database Synchronization

Although real-time updates are handled through RESTful APIs and WebSockets, simultaneous high-frequency updates from multiple devices could cause slight delays in syncing, especially under network congestion.

10.3 Design and Architecture Challenges

10.3.1 Multi-Branch Data Isolation

Ensuring strict separation between pharmacy branches while maintaining a centralized backend architecture required careful database design to avoid data leakage across branches.

10.3.2 Real-Time Collaboration

Implementing real-time collaboration (e.g., multiple workers checking out simultaneously) required careful concurrency management and validation at the server level to avoid conflicts.

10.3.3 Barcode/Visual Redundancy

The absence of standardized barcode systems across all drugs pushed the system to rely more heavily on computer vision, which is inherently probabilistic and can't match the precision of traditional barcodes, whoever it is faster.

10.4 User Experience Challenges

10.4.1 Training Staff to Use the App

Workers unfamiliar with Android or mobile apps required simple and clear UI to interact confidently with features like object detection and drug searching. A usability-first UI design update can help minimize confusion.

10.4.2 Receipt Return/Replacement Logic

Implementing receipt replacement required extra validation logic to prevent fraud or duplication. Additional backend logic was added to verify user authorization and receipt timestamps.

10.5 Operational and Deployment Limitations

10.5.1 Internet Dependency

The system relies on stable internet connectivity to function properly, especially for features like drug recognition and real-time data syncing. Offline capabilities were limited to basic caching.

10.5.2 Scalability Concerns

While the system was designed for small to medium-sized pharmacies, large-scale operations with hundreds of concurrent users would require more robust cloud infrastructure and load balancing strategies.

10.5.3 Security Risks

Although authenticated APIs and database role separation were used, further enhancements such as two-factor authentication (2FA), encrypted data transmission, and session token hardening are recommended for future versions.

11 Chapter Eleven: Conclusion and Future Work

11.1 Conclusion

The decentralized pharmacy management system developed in this project—FastAF—successfully addresses a significant operational bottleneck found in traditional pharmacy workflows: the single point of failure at the cashier terminal. By empowering pharmacy staff to perform the cashiering process directly through their Android devices, the system enhances flexibility, minimizes downtime, and increases sales throughput, particularly during peak hours.

The project comprises two main components:

- An **Android mobile application** that enables pharmacy workers to perform sales, manage receipts, handle deliveries, and visually detect drugs using a YOLO-based object detection model.
- A **web-based dashboard** that allows pharmacy managers and administrators to create pharmacy branches, register employees, track sales analytics, manage inventory, monitor drug expiries, and oversee operational activities in real time.

The combination of AI-driven drug recognition using Yolo11 model, real-time database synchronization, and multi-user collaboration across mobile and web platforms provides a modern and scalable solution for pharmacies. The use of tools like CameraX, Retrofit, MVVM, and WebSocket communication ensures a responsive and efficient user experience.

This system contributes to the digitization and decentralization of pharmacy operations, making them more resilient, efficient, and intelligent.

11.2 Future Work

While the current system demonstrates strong functionality, several enhancements can be made to improve performance, usability, and scalability:

Dataset Expansion for Drug Detection

The current YOLO model sometimes fails to differentiate between visually similar drug packages due to a limited training dataset. Future work should focus on:

- Expanding the dataset to include more drugs, multiple packaging variants, and more visual angles.

- Adding OCR-based drug name recognition as a fallback mechanism when visual features are ambiguous or to validate the yolo prediction.

Offline Mode with Sync Queue

Implementing offline capabilities for the Android app will improve usability in areas with poor or intermittent internet. This includes:

- Local storing of the sales during offline mode.
- Implement small model to identify drugs for sales (e.g., small OpenCV model deployed locally on the app to detect barcodes)
- Queuing API requests for synchronization once connectivity is restored.

Improved Security

The system can be further secured with:

- Two-Factor Authentication (2FA) for admin and manager logins.
- JWT refresh tokens and expiry policies.
- End-to-end encryption for sensitive transactions and receipt handling.

Cross-Platform Support

Developing an iOS version of the mobile app would expand the system's usability across more devices and user preferences.

Smart Inventory Prediction

Integrate predictive analytics to:

- Forecast drug demand based on sales trends.
- Automatically suggest reorders or re-stocking quantities.
- Warn about unusual drops in sales for specific items.

Role-Based Access Control (RBAC) Enhancements

Introduce more granular permissions and roles with customized access levels, enhancing security and accountability for bigger pharmacies.

References

1. **Higher Technology Institute.** (2025). *Computer Science Department*. 10th of Ramadan City.
2. **YOLOv11 Documentation.** (2023). *Real-Time Object Detection Model*. Retrieved from [YOLO Official Website](#)
3. **Spring Boot Documentation.** (2023). *Building RESTful APIs with Spring Boot*. Retrieved from [Spring Official Website](#)
4. **Android Developers.** (2023). *CameraX API for Android*. Retrieved from [Android Developer Website](#)
5. **FastAPI Documentation.** (2023). *WebSocket API Development*. Retrieved from [FastAPI Official Website](#)
6. **PostgreSQL Documentation.** (2023). *Database Management and Optimization*. Retrieved from [PostgreSQL Official Website](#)
7. **Redis Documentation.** (2023). *Caching Strategies and Implementation*. Retrieved from [Redis Official Website](#)
8. **Roboflow.** (2023). *Dataset Annotation and Augmentation Tools*. Retrieved from [Roboflow Official Website](#)
9. **Tailwind CSS Documentation.** (2023). *Utility-First CSS Framework*. Retrieved from [Tailwind CSS Official Website](#)
10. **React Documentation.** (2023). *Building User Interfaces with React*. Retrieved from [React Official Website](#)
11. **IEEE.** (2019). *Improving Drug Dispensation Using Smart POS Terminals*. IEEE Xplore.
12. **Health IT Analytics.** (2021). *Pharmacy POS System Downtime and Its Impact*. Health IT Analytics Journal.

13. **Elsevier.** (2022). *Mobile Applications for Pharmacy Inventory Management*. Elsevier Journal of Healthcare Technology.
14. **IJCA.** (2020). *Design of an Expiry Management System for Pharmacies*. International Journal of Computer Applications.
15. **Google Cloud.** (2023). *Deploying AI Models on Cloud VMs*. Retrieved from [Google Cloud Documentation](#)
16. **Docker Documentation.** (2023). *Containerization and Deployment*. Retrieved from [Docker Official Website](#)
17. **GitHub Actions.** (2023). *CI/CD Pipeline Automation*. Retrieved from [GitHub Docs](#)
18. **Azure Documentation.** (2023). *Cloud Resources and Scalability*. Retrieved from [Microsoft Azure Official Website](#)
19. **OpenCV Documentation.** (2023). *Image Processing and Computer Vision*. Retrieved from [OpenCV Official Website](#)
20. **ONNX Runtime.** (2023). *Optimized Model Inference*. Retrieved from [ONNX Runtime Official Website](#)